

Fundamentals of Natural Language Processing

Course material

M1 GAMMA & L2 LMAD — Data Science

Aymen Ben Brik

May 6, 2026

Course Fundamentals of Natural Language Processing
Audience M1 GAMMA
L2 LMAD — Data Science
Author Aymen Ben Brik
Repository <https://github.com/Aymenbenbrik/FundamentalsNLP>

Foreword

This document is the unified support for an introductory Natural Language Processing course aimed at first-year M1 (GAMMA) and second-year L2 (LMAD Data Science) students. It assumes basic Python programming, working knowledge of machine learning and linear algebra (vectors, dot products, distances), and a willingness to read mathematical definitions.

Pedagogy. Each chapter follows a unified structure: a concrete *motivation*, an *approach by problem*, formal *definitions and theorems* with proofs where appropriate, *worked examples* of ascending difficulty, integrated *Python code* (`scikit-learn`, `NLTK`, `gensim`, `spaCy`, `transformers`), and *exercises*. Many examples and exercises are framed in a Tunisian or multilingual context to ensure relevance to the target audience.

Companion material. Four Jupyter lab notebooks accompany the chapters in the `Notebooks/` folder of the GitHub repository:

- `lab01-tokenization.ipynb` — whitespace, regex, BPE from scratch, BERT vs GPT-2 tokenizers.
- `lab02-bow-tfidf-classifier.ipynb` — Bag-of-Words, TF-IDF, logistic-regression classifier with feature interpretation.
- `lab03-word2vec-embeddings.ipynb` — skip-gram / CBOW, similarity, analogies, PCA visualization.
- `lab04-evaluation-metrics.ipynb` — classification metrics, ROC / PR curves, perplexity, BLEU, ROUGE.

Learning objectives. By the end of this course, the student should be able to:

1. Define NLP and identify its three core challenges (ambiguity, context, variability).
2. Recognize the major NLP tasks and their real-world applications.
3. Situate NLP within the broader AI / ML / DL landscape and identify the dominant learning paradigms.
4. Distinguish discriminative from generative modelling approaches.
5. Trace the historical evolution of NLP from rule-based systems to large language models.
6. Apply tokenization strategies appropriate to a given task and language.

7. Build sparse (BoW, TF-IDF) and dense (Word2Vec, GloVe, FastText) text representations.
8. Compute and interpret cosine, dot-product and Euclidean similarities.
9. Evaluate NLP systems with task-appropriate metrics (Precision, Recall, F_1 , perplexity, BLEU, ROUGE) and recognise their pitfalls.

Contents

Foreword	1
1 The NLP Landscape	6
1.1 What is NLP? Definition, scope, and core challenges	6
1.1.1 Motivation	6
1.1.2 Approach by problem: the headline that everyone misreads	6
1.1.3 Definition and scope	7
1.1.4 Why natural language is hard	7
1.1.5 Worked examples	8
1.1.6 Python snippet: probing word senses	9
1.1.7 Exercises	10
1.2 Major NLP tasks and applications	10
1.2.1 Motivation	10
1.2.2 Approach by problem: one dataset, many tasks	10
1.2.3 A taxonomy by input–output structure	11
1.2.4 Text classification	11
1.2.5 Sequence labelling	12
1.2.6 Sequence-to-sequence	12
1.2.7 Question answering	13
1.2.8 Open-ended generation	13
1.2.9 Worked examples	14
1.2.10 Python snippet: spaCy pipeline	15
1.2.11 Exercises	15
1.3 NLP within the AI ecosystem	16
1.3.1 Motivation	16
1.3.2 Approach by problem: the same data, three paradigms	16
1.3.3 Situating NLP: AI, ML, DL	16
1.3.4 The four ingredients of every learned NLP system	17
1.3.5 Three learning paradigms	17
1.3.6 Comparison table	18
1.3.7 Worked examples	18
1.3.8 Python snippet: train your first text classifier in 10 lines	20
1.3.9 Exercises	20
1.4 Discriminative vs. generative models	21
1.4.1 Motivation	21
1.4.2 Approach by problem: classify or generate?	21
1.4.3 Discriminative models	21
1.4.4 Generative models	22

1.4.5	Comparison and the modern blur	22
1.4.6	The Bayes bridge	23
1.4.7	Worked examples	23
1.4.8	Python snippet: probabilistic interpretation	24
1.4.9	Exercises	25
1.5	Historical evolution: from rules to large language models	26
1.5.1	Motivation	26
1.5.2	Approach by problem: how would you build a chatbot in 1966?	26
1.5.3	Era 1 — Rule-based systems (1950s–1980s)	26
1.5.4	Era 2 — Statistical NLP (1990s–2000s)	27
1.5.5	Era 3 — Neural networks and word embeddings (2013–2017)	28
1.5.6	Era 4 — Transformers and large language models (2017–present)	28
1.5.7	Synthetic comparison	29
1.5.8	The recurring pattern	29
1.5.9	Worked examples	29
1.5.10	Python snippet: a four-line bigram language model	30
1.5.11	Exercises	31
2	Text Representation and Evaluation	32
2.1	Tokenization fundamentals	32
2.1.1	Motivation	32
2.1.2	Approach by problem: how many tokens are in this sentence?	32
2.1.3	Definition and the tokenization pipeline	33
2.1.4	Three levels of granularity	33
2.1.5	Pre-processing operations	34
2.1.6	Byte-Pair Encoding (BPE) in detail	34
2.1.7	Multilingual and dialectal challenges	35
2.1.8	Worked examples	35
2.1.9	Python snippet: comparing tokenizers	36
2.1.10	Exercises	36
2.2	Sparse representations: Bag-of-Words and TF–IDF	37
2.2.1	Motivation	37
2.2.2	Approach by problem: are these two reviews similar?	37
2.2.3	Vector space models	37
2.2.4	Bag-of-Words	38
2.2.5	The document–term matrix	38
2.2.6	Limitations of Bag-of-Words	38
2.2.7	TF–IDF	39
2.2.8	TF–IDF as a probabilistic measure	39
2.2.9	Worked examples	40
2.2.10	Python snippet: from text to TF–IDF in 5 lines	40
2.2.11	Exercises	41
2.3	Dense representations: Word2Vec, GloVe, FastText	42
2.3.1	Motivation	42
2.3.2	Approach by problem: where does meaning live?	42
2.3.3	The distributional hypothesis	42
2.3.4	Word2Vec	42
2.3.5	GloVe	43

2.3.6	FastText	44
2.3.7	Vector arithmetic and the analogy task	44
2.3.8	Limitations: static, monolingual, biased	44
2.3.9	Toward contextual embeddings	45
2.3.10	Worked examples	45
2.3.11	Python snippet: train and explore Word2Vec	46
2.3.12	Exercises	46
2.4	Semantic similarity measures	47
2.4.1	Motivation	47
2.4.2	Approach by problem: ranking by similarity	47
2.4.3	Three classical measures	47
2.4.4	Properties of cosine similarity	48
2.4.5	When to use which	49
2.4.6	Applications	49
2.4.7	The retrieval pattern	49
2.4.8	Worked examples	49
2.4.9	Python snippet: cosine, Euclidean, and FAISS	50
2.4.10	Exercises	51
2.5	Evaluation metrics	52
2.5.1	Motivation	52
2.5.2	Approach by problem: the misleading 95% accuracy	52
2.5.3	Classification metrics	52
2.5.4	Generation metrics	54
2.5.5	Summary table	55
2.5.6	Worked examples	55
2.5.7	Python snippet: classification + generation metrics	56
2.5.8	Exercises	57

Chapter 1

The NLP Landscape

This opening chapter situates Natural Language Processing as a field: what problem it tries to solve, what makes that problem fundamentally hard, what canonical tasks define the discipline, and how its methods have evolved from hand-written rules to billion-parameter language models. Throughout, the goal is to develop a precise mental map you can return to as more advanced techniques are introduced.

1.1 What is NLP? Definition, scope, and core challenges

1.1.1 Motivation

Computers are excellent with *structured* data: spreadsheets, SQL tables, signed numerical vectors. They struggle with *unstructured* human language — the messy, fluid, ambiguous medium humans use to communicate. Yet a vast majority of the world’s data exists in textual form: emails, chat messages, news articles, scientific papers, social-media posts, customer reviews, medical records, contracts. Building machines that can read, summarize, translate, search, and converse in natural language unlocks nearly every modern AI product, from search engines and recommendation systems to clinical decision-support tools and large language model assistants. *Natural Language Processing* (NLP) is the field that develops and studies the algorithmic foundations of this capability.

1.1.2 Approach by problem: the headline that everyone misreads

Problem. Consider the (real) news headline:

“Stolen painting found by tree.”

List every grammatically valid reading. How many require world knowledge to discard? How could an algorithm resolve the intended one?

Solution sketch. The headline admits at least two parses:

1. “Stolen painting [found by [tree]]” — a tree found the painting.
2. “Stolen painting [found [by tree]]” — the painting was found beside a tree.

Reading 1 is grammatically valid but pragmatically absurd: trees do not perform searches. A native speaker rules it out using world knowledge (trees are inanimate). A computer that does *not* have such knowledge cannot rule it out without an explicit mechanism — whether a hand-coded rule, statistical prior, or learned representation. Multiplying this difficulty across millions of sentences, in dozens of languages, is the central challenge NLP must address.

1.1.3 Definition and scope

Definition 1.1.1 (Natural Language Processing). **Natural Language Processing** (NLP) is the subfield of artificial intelligence concerned with enabling computer systems to process, understand, and generate human language. It sits at the intersection of *linguistics* (the scientific study of language), *computer science* (algorithms and data structures) and *machine learning* (data-driven model building).

It is convenient to organise the field along three complementary axes:

- **Natural Language Understanding (NLU)** — extracting meaning, structure, sentiment, intent, or entities from text. *Inputs* are texts, *outputs* are typically structured representations (labels, parse trees, knowledge-graph triples).
- **Natural Language Generation (NLG)** — producing fluent and coherent natural-language outputs from structured inputs (database rows, dialogue states, summaries of long documents).
- **Interaction** — dialogue systems and conversational agents, where understanding and generation operate in a closed loop with a user.

1.1.4 Why natural language is hard

Three orthogonal sources of difficulty pervade every NLP system.

1. Ambiguity

A single linguistic expression can correspond to multiple meanings or structures. Ambiguity arises at every level of analysis.

- **Lexical ambiguity** (polysemy or homonymy). A single word denotes several distinct concepts.
 - *bank*: financial institution VS. edge of a river.
 - *date*: calendar day VS. romantic appointment VS. fruit.
- **Syntactic ambiguity**. A single string can be parsed by several distinct grammatical structures.
 - “*I saw the man with the telescope.*” — I used the telescope, or the man had a telescope?
- **Semantic ambiguity**. A single syntactic structure admits several meanings.
 - “*The chicken is ready to eat.*” — the chicken will eat, or will be eaten?

- **Pragmatic ambiguity.** The intended communicative function is underdetermined by the literal content.
 - “*Can you pass the salt?*” is a polite request, not a yes/no inquiry about the listener’s capabilities.

2. Context dependence

The meaning of an expression cannot be computed from its surface form alone — it depends on its surrounding linguistic context and on world knowledge.

- “*Apple released a new product.*” (the company) vs. “*Apple is healthy.*” (the fruit). The same string, two referents.
- Pronoun resolution: “*John told Bill that he had failed the exam.*” — who failed? Either reading is grammatical; only context decides.
- Sarcasm and irony: “*Oh great, another delay.*” may carry the opposite of its literal meaning.

3. Variability

Many distinct surface forms encode the same meaning, and natural language permits substantial creativity.

- **Synonymy and paraphrasing:** “*happy*”, “*joyful*”, “*content*”, “*elated*” are near-synonyms with different connotations.
- **Spelling and orthographic variation:** “*colour*” / “*color*”, typos, leetspeak, abbreviations.
- **Cross-domain shifts:** medical, legal, social-media, and academic text differ in vocabulary, syntax, and conventions.
- **Multilingualism and code-switching:** “*J’ai*” un meeting demain — many speakers freely mix languages.

Remark 1.1.1. These three challenges combine multiplicatively. An algorithm that handles ambiguity but not context will fail on the simplest pronoun resolution; one that handles context but not variability will not generalize from training to deployment data.

1.1.5 Worked examples

Example 1 (Lexical ambiguity — Easy). Identify the polysemous word in the following sentence and give at least two distinct readings:

She left her glasses on the table.

Solution. The polysemous word is *glasses*, which may refer to:

- eyewear (a pair of corrective spectacles), or
- drinking vessels (e.g. wine glasses).

World knowledge — e.g. that the speaker is myopic, or that the table held a meal — is needed to disambiguate.

Example 2 (Syntactic ambiguity — Medium). Resolve the ambiguity of “*He hit the boy with the bat*” under the following two contexts:

Context A: The previous sentence was “*He grabbed a wooden bat from the dugout.*”

Context B: The previous sentence was “*He met the boy with the bat at the zoo.*”

Solution. In context A, “*with the bat*” attaches to the verb *hit* (instrumental adjunct): the bat is the weapon. In context B, “*with the bat*” attaches to the noun *boy* (modifier): the boy carried a bat (the animal). Same surface form, two parse trees, two meanings; the intended one is selected only via context.

Example 3 (Pragmatics over literal content — Hard). The utterance “*It is cold in here.*” can play several pragmatic roles. List three plausible interpretations and explain when each applies.

Solution.

1. **Literal assertion:** a factual statement about the temperature, expecting acknowledgement.
2. **Indirect request:** an implicit suggestion to close a window, turn up heating, or hand the speaker a sweater (when the speaker has the social standing to make a request).
3. **Sarcasm:** in a room that is unmistakably warm, the same string may serve as ironic commentary on, e.g., an awkward silence.

Selecting the correct reading requires modelling the speaker, the listener, the physical situation, and the broader cultural conventions of indirect speech — topics studied under the name of *pragmatics*.

1.1.6 Python snippet: probing word senses

The Princeton WordNet lexical database, accessible through NLTK, allows us to enumerate distinct senses of a polysemous word.

```
1 import nltk
2 nltk.download('wordnet', quiet=True)
3 from nltk.corpus import wordnet as wn
4
5 def show_senses(word):
6     senses = wn.synsets(word)
7     print(f"'{word}' has {len(senses)} sense(s) in WordNet:")
8     for syn in senses[:6]:
9         print(f"{syn.name():<20s} {syn.definition()}")
10
11 show_senses("bank")
12 show_senses("date")
13 show_senses("light")
```

Running this snippet illustrates that even commonplace words carry many distinct senses — the raw input to any system that aspires to “understand” text.

1.1.7 Exercises

1. Find three polysemous English words used in everyday speech, and list at least two distinct senses of each. For each pair of senses, propose a sentence in which only one of them is felicitous.
2. For the sentence “*The duchess was entertaining last night,*” identify two grammatical readings and explain how the ambiguity is structured (lexical or syntactic).
3. Find a real-world headline (e.g. from the Wikipedia article on “Crash blossom” or from a news aggregator) that exhibits at least one form of ambiguity. Classify the ambiguity (lexical / syntactic / semantic / pragmatic) and propose a context that resolves it.
4. Discuss: how might *world knowledge* help a system disambiguate “*I deposited the money in the bank*”? What form would such knowledge take, and where might it be obtained at scale?
5. (Synthesis.) Suppose you must build a customer-service chatbot for a Tunisian e-commerce site that receives messages mixing French, Arabic and English. Identify, for each of the three challenges (*ambiguity*, *context*, *variability*), one concrete obstacle your system will face that an English-only system would not.

1.2 Major NLP tasks and applications

1.2.1 Motivation

NLP is not a single problem; it is a constellation of problems, each with its own *input format*, *output structure*, *evaluation metric*, and *characteristic methods*. A sentiment classifier and a machine-translation system both consume text, but they belong to different computational categories and therefore demand different architectures and training data. Practitioners must develop a clear taxonomy of NLP tasks for three practical reasons:

1. to *frame* a real-world business problem as the right computational task;
2. to *select* the appropriate model family (classification vs. sequence labelling vs. generation);
3. to *evaluate* models with metrics that match the task — accuracy is meaningful for classification but disastrous as a sole metric for translation.

1.2.2 Approach by problem: one dataset, many tasks

Problem. You receive a corpus of 50,000 customer reviews of a hotel chain. Each review is a paragraph of free text, sometimes with a star rating, sometimes not. Your manager asks: “What can we do with this data?” List at least five distinct NLP problems you could solve from *the same* raw input.

Solution sketch. The same corpus can power multiple downstream tasks:

1. **Sentiment analysis** — classify each review as positive, negative, or neutral.
2. **Topic classification** — assign reviews to themes (food, cleanliness, location, staff).

3. **Aspect-based sentiment** — extract sentiment per aspect (“rooms: positive, breakfast: negative”).
4. **Named entity recognition** — extract place names, dates, hotel branches mentioned.
5. **Summarization** — produce a one-sentence digest of each review.
6. **Translation** — translate Arabic and French reviews into English for cross-lingual analytics.
7. **Question answering** — support a query interface (“What do guests say about the pool?”).

Each item belongs to a different computational category, requires different annotated data, and is evaluated with different metrics. The remainder of this section gives a structured tour of these categories.

1.2.3 A taxonomy by input–output structure

We organise NLP tasks by what the model *consumes* and *produces*.

Category	Input	Output
Text classification	text	one (or several) discrete labels
Sequence labelling	text	one label per token
Sequence-to-sequence	text	a different text
Question answering	question + context	answer (span or text)
Open-ended generation	prompt	arbitrary text

The categories form a rough *spectrum of output complexity*, from a single integer (classification) to a free-form sequence (generation). Each step adds expressive power and evaluation difficulty.

1.2.4 Text classification

Definition 1.2.1 (Text classification). Given an input text x drawn from a domain $\mathcal{X}$ and a finite label set $\mathcal{Y} = \{y_1, \dots, y_K\}$, learn a function $f : \mathcal{X} \rightarrow \mathcal{Y}$ (single-label) or $f : \mathcal{X} \rightarrow \mathcal{P}(\mathcal{Y})$ (multi-label) that assigns labels to texts.

Canonical applications.

- **Sentiment analysis** — $\mathcal{Y} = \{\text{positive, negative, neutral}\}$. Powers product-review analytics, social-media monitoring, financial-sentiment trading signals.
- **Spam detection** — $\mathcal{Y} = \{\text{spam, ham}\}$. The earliest commercially deployed NLP application at scale.
- **Topic classification** — assign a document to one of several thematic categories (politics, sports, technology, ...).
- **Intent detection** — in chatbots, classify a user utterance into pre-defined intents (BOOK-FLIGHT, CANCEL-ORDER, ASK-STATUS).
- **Toxicity / hate-speech detection** — moderate user-generated content on social platforms.

Standard metrics. Accuracy, Precision, Recall, F_1 (Section 2.5.3 will define them precisely).

1.2.5 Sequence labelling

Definition 1.2.2 (Sequence labelling). Given a sequence of tokens $x = (x_1, x_2, \dots, x_n)$, predict a sequence of labels $y = (y_1, y_2, \dots, y_n)$ of the same length, where each y_i belongs to a finite label set $\mathcal{Y}$.

The output dimension grows with the input, in contrast to classification.

Canonical applications.

- **Part-of-speech (POS) tagging** — label every token with its grammatical role: *noun, verb, adjective, ...*

“Apple/NOUN released/VERB a/DET new/ADJ phone/NOUN.”

- **Named Entity Recognition (NER)** — label tokens that participate in named entities (persons, organizations, locations, dates) with their type and span.

“Apple/B-ORG was/O founded/O by/O Steve/B-PER Jobs/I-PER in/O Cupertino/B-LOC.”

The B-/I-/O scheme distinguishes *Begin* of an entity, *Inside*, and *Outside*.

- **Chunking and shallow parsing** — group tokens into noun phrases, verb phrases.
- **Slot filling** — in dialogue systems, identify the DATE, ORIGIN, DESTINATION slots in user utterances.

High-impact applications. Pharmacovigilance (drug-name extraction from medical records), legal-document analysis (clause boundaries), bioinformatics (gene and protein-name recognition), search-engine query understanding.

1.2.6 Sequence-to-sequence

Definition 1.2.3 (Sequence-to-sequence task). Given a source sequence $x = (x_1, \dots, x_n)$, produce a target sequence $y = (y_1, \dots, y_m)$ where m may differ from n and the target vocabulary may differ from the source.

This category is the most general; it subsumes any task where the output is itself a piece of text.

Canonical applications.

- **Machine translation** — map a sentence in source language L_1 to its translation in target language L_2 . Output length, word order, and idioms differ from the source.
- **Abstractive summarization** — produce a short summary that need not reuse the exact wording of the input. Contrast with *extractive* summarization, which selects sentences from the input verbatim.

- **Paraphrasing and style transfer** — rewrite a text with the same meaning but different surface form (e.g. formal to informal).
- **Grammar correction** — map an error-laden sentence to its corrected version.

Standard metrics. BLEU (translation), ROUGE (summarization). Both are studied formally in Section 2.5.4.

1.2.7 Question answering

Definition 1.2.4 (Question answering). Given a question q and (optionally) a context document or knowledge base c , produce an answer a to the question.

QA is conceptually a special case of sequence-to-sequence but is distinguished by the structure of the input (question + context) and by several distinct sub-paradigms:

- **Extractive QA.** The answer is a contiguous span of the context document. The model’s job is to predict the start and end positions. Typical of reading-comprehension benchmarks (SQuAD).
- **Closed-book QA.** No context is provided; the model must rely on knowledge stored in its parameters.
- **Open-domain QA.** The model first *retrieves* relevant documents from a large corpus, then reads them to extract the answer. This is the foundation of the *retrieval-augmented generation* (RAG) pattern.
- **Generative QA.** The answer is generated freely, possibly synthesising information from the context.

Industrial deployments. Customer-service chatbots (intent+slot extraction), Web search (featured snippets), enterprise document search.

1.2.8 Open-ended generation

Definition 1.2.5 (Open-ended generation). Given a prompt x , produce a continuation y that is fluent, coherent, and consistent with x but otherwise underdetermined.

This is the regime of large language models. The task is intentionally under-specified: many distinct continuations may be acceptable. Evaluation is therefore especially challenging.

Applications. Creative writing assistance, code generation (autocomplete and full-function synthesis), dialogue agents, automatic e-mail drafting, automated report generation from structured data.

1.2.9 Worked examples

Example 1 (Task framing — Easy). You are given the following row from a dataset of medical encounter notes:

Patient (45 y, F) reports persistent headache and was prescribed Paracetamol 500 mg twice a day.

Identify two distinct NLP tasks that operate on this single sentence, naming the appropriate category for each.

Solution.

- **Named Entity Recognition** (sequence labelling): extract [Drug: Paracetamol], [Dose: 500 mg], [Frequency: twice a day], [Symptom: headache].
- **Text classification**: classify the note’s clinical specialty (e.g. *neurology*).

Example 2 (Choosing the right output structure — Medium). You wish to build a system that, given a research-paper abstract, produces “the three most important takeaways”. Should you frame this as a sequence-to-sequence task or as a classification task? Justify.

Solution. A classification task forces the output into a finite, pre-defined set of labels — but the “takeaways” are free-form sentences whose content depends on each abstract. The natural framing is therefore *abstractive summarization* (sequence-to-sequence), with an additional length constraint that forces the output to consist of three short sentences. A pre-defined-label classification approach would be tractable only if you replaced “the three most important takeaways” by “one of K predefined research-area tags,” which is a substantively different problem.

Example 3 (Decomposing a complex query — Hard). A user asks the chatbot of a Tunisian airline: “*I bought a ticket to Paris last week, but my passport just expired. What should I do?*” Decompose this single utterance into the NLP sub-tasks the system must execute to respond usefully.

Solution.

1. **Intent detection** (classification): the intent is approximately HELP-WITH-DOCUMENT-ISSUE or CHANGE-BOOKING.
2. **Slot filling / NER** (sequence labelling): extract [Destination: Paris], [BookingDate: last week], [DocumentIssue: expired passport].
3. **Knowledge retrieval** (open-domain QA): fetch the airline’s policy on travel-document issues.
4. **Generation**: synthesise an empathetic, accurate, and fluent response that explains the available options (refund, ticket transfer, document renewal).

A modern dialogue system performs these steps either as separate components in a pipeline, or jointly inside a single large language model that is prompted with the user message and policy documents.

1.2.10 Python snippet: spaCy pipeline

The library spaCy ships pre-trained models that perform several of the tasks above in a single forward pass.

```
1 # pip install spacy && python -m spacy download en_core_web_sm
2 import spacy
3 nlp = spacy.load("en_core_web_sm")
4 text = ("Apple was founded by Steve Jobs in Cupertino in 1976. "
5         "Today, the company is worth over $3 trillion.")
6 doc = nlp(text)
7
8 print("Tokens with POS tags:")
9 for token in doc[:10]:
10     print(f"_{token.text:<12s}_{token.pos_:<6s}_{token.dep_}")
11
12 print("\nNamed entities:")
13 for ent in doc.ents:
14     print(f"_{ent.text:<20s}_{ent.label_}")
```

A single call returns tokenisation, POS tags, dependency parse, and named entities — a concrete instantiation of the sequence-labelling and classification ideas of this section.

1.2.11 Exercises

1. For each of the following, identify the most appropriate NLP task category (classification, sequence labelling, sequence-to-sequence, QA, generation):
 - (a) Detecting whether an SMS is fraudulent.
 - (b) Translating a Tunisian Arabic Facebook post into French.
 - (c) Extracting the names of all medications mentioned in a pharmacist's note.
 - (d) Producing a one-paragraph summary of a court ruling.
 - (e) Determining the writer's age range from a blog post.
2. For *aspect-based sentiment analysis* (e.g. "the food was great but the service was slow"), argue that the natural framing combines two distinct task categories. Which two?
3. Choose one industry (healthcare, finance, legal, education) and list three concrete NLP applications it uses. For each, identify the input, the output, and the dominant challenge from Section 1.1.
4. Discuss: should a search-engine query understanding system be framed as classification, sequence labelling, or generation? Defend your choice and identify trade-offs.
5. (Open-ended.) Sketch a pipeline that, given a Tunisian e-commerce review written in code-mixed French–Arabic, produces (a) an overall sentiment, (b) a list of mentioned product aspects, and (c) an English summary. Identify each NLP task involved and the order of execution.

1.3 NLP within the AI ecosystem

1.3.1 Motivation

Modern NLP is, in practice, a sub-discipline of *machine learning* (ML), which is itself a sub-discipline of *artificial intelligence* (AI). Reading any contemporary NLP paper requires fluency in concepts that originate outside NLP proper: gradient descent, regularization, train/validation/test splits, the bias-variance trade-off, the empirical risk minimization principle. Conversely, modern NLP has pushed the entire ML field forward: Transformers, scaling laws, instruction tuning, chain-of-thought prompting — innovations born in NLP now drive computer vision, speech, and robotics. Understanding where NLP sits in the broader AI landscape is therefore essential, both to use existing tools correctly and to situate new methods as they appear.

1.3.2 Approach by problem: the same data, three paradigms

Problem. Suppose you have collected 20,000 tweets about Tunisian banks. Describe how you could exploit this single corpus under three radically different learning regimes:

1. one in which every tweet has a hand-annotated sentiment label;
2. one in which no annotations are available;
3. one in which no annotations are available, but you wish to learn *useful representations* of words that subsequent tasks can re-use.

Solution sketch.

1. With labels: train a *supervised* sentiment classifier — input tweet, output positive / negative / neutral.
2. Without labels: *cluster* the tweets by topic; discover latent themes (*unsupervised*).
3. Without labels but for representation learning: train a model that, for each word, predicts the surrounding words in a sliding window (*self-supervised*). The model never receives “labels” from a human; it manufactures its own training signal from the data itself. This is exactly the principle behind Word2Vec (Section 2.3) and, at much larger scale, behind every Large Language Model.

1.3.3 Situating NLP: AI, ML, DL

Definition 1.3.1 (Artificial intelligence). **Artificial intelligence** (AI) is the area of computer science concerned with building systems that perform tasks that, when performed by humans, would be said to require *intelligence* — such as perception, reasoning, learning, language understanding, problem-solving, and action under uncertainty.

Definition 1.3.2 (Machine learning). **Machine learning** (ML) is the sub-area of AI in which the system’s behaviour is *induced from data* rather than directly programmed by humans. A learning algorithm transforms a dataset D into a predictor f .

Definition 1.3.3 (Deep learning). **Deep learning** (DL) is the sub-area of ML that uses *deep neural networks* — parameterised functions composed of many layers of differentiable transformations — as the predictor family. Deep learning powers most state-of-the-art NLP, computer-vision, and speech systems.

The relationships between these fields are strictly inclusive: $DL \subset ML \subset AI$. NLP is a *domain of application*: it draws on rule-based AI for early systems, on statistical ML for the n-gram era, and on deep learning for everything since 2013.

1.3.4 The four ingredients of every learned NLP system

Definition 1.3.4 (Learning ingredients). A supervised (or self-supervised) learning system is fully specified by the quadruple

$$(D, \mathcal{F}, \mathcal{L}, \mathcal{O})$$

where:

- **Data** $D = \{(x_i, y_i)\}_{i=1}^N$ — a set of training examples, each consisting of an input and a target;
- **Model family** $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$ — a parameterised set of functions from inputs to predictions;
- **Loss** $\mathcal{L}(\hat{y}, y)$ — a scalar measure of how badly a prediction $\hat{y}$ matches the target y ;
- **Optimizer** $\mathcal{O}$ — an algorithm that updates θ to reduce the average loss on the data.

Definition 1.3.5 (Empirical risk minimization). The dominant training principle is **empirical risk minimization** (ERM): seek the parameters θ^* that minimise the average loss on the training data,

$$\theta^* \in \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_\theta(x_i), y_i).$$

Computing θ^* exactly is intractable for neural networks; the optimizer $\mathcal{O}$ (typically Stochastic Gradient Descent or its variants Adam, AdamW) approximates the minimum by iterative updates.

Remark 1.3.1. The four ingredients are useful as a debugging checklist. If a deployed system underperforms, the cause is almost always traceable to one of them: poor data quality, an inadequate model family, a mismatched loss, or an unstable optimization procedure.

1.3.5 Three learning paradigms

Supervised learning

Definition 1.3.6 (Supervised learning). Each training example is a pair (x, y) where y is provided by an external annotator (the “supervision”). The learner’s goal is to predict y from x on unseen inputs.

NLP examples. Sentiment classification, machine translation (parallel sentences as labels), POS tagging, NER. The bottleneck is annotation cost: linguistically reliable labels typically cost \$0.10–\$10 per example.

Unsupervised learning

Definition 1.3.7 (Unsupervised learning). Training examples consist only of inputs x , with no labels. The goal is to discover *structure* in the data: clusters, latent variables, low-dimensional manifolds.

NLP examples. Topic modelling (LDA), document clustering, anomaly detection in log files. Output is structural rather than predictive.

Self-supervised learning

Definition 1.3.8 (Self-supervised learning). Training examples consist only of inputs x , but the learner constructs a *pretext task* from the data itself — for example, predicting a hidden portion of x from the rest. Labels are manufactured automatically from the data, eliminating the need for human annotation.

Canonical NLP pretext tasks.

- **Causal language modelling:** given the prefix $w_1, \dots, w_{t-1}$, predict the next token w_t . Used by GPT-style models.
- **Masked language modelling:** randomly mask $\sim 15\%$ of tokens, predict them from the surrounding context. Used by BERT.
- **Skip-gram and CBOW:** predict context words from a centre word, or vice-versa. Used by Word2Vec.

Remark 1.3.2 (Why self-supervision dominates modern NLP). Self-supervised pretraining converts *any* large unlabelled text corpus (a substantial fraction of the entire Web) into a training signal. The resulting models acquire general-purpose linguistic competence that is then *transferred* to downstream tasks via fine-tuning or prompting. This decoupling of *representation learning* (cheap, vast) and *task adaptation* (cheap, small) explains the dramatic progress of NLP since 2018.

1.3.6 Comparison table

Paradigm	Labels?	Goal	NLP example
Supervised	Provided by humans	Predict y from x	Spam detection
Unsupervised	None	Discover structure	Topic clustering
Self-supervised	Manufactured from x	Learn representations	Next-token prediction (LLMs)

1.3.7 Worked examples

Example 1 (Identifying the paradigm — Easy). For each of the following NLP tasks, identify the dominant learning paradigm:

1. Detecting whether a customer review is positive or negative, given a labelled dataset of 10,000 reviews.
2. Discovering the principal themes in 1 million unlabelled news articles.
3. Pretraining a 1B-parameter language model on 300 GB of Web text.

Solution. (1) Supervised. (2) Unsupervised (typically LDA or K-means on document embeddings). (3) Self-supervised (next-token or masked-token prediction).

Example 2 (The four ingredients in action — Medium). Spell out $(D, \mathcal{F}, \mathcal{L}, \mathcal{O})$ for a logistic-regression sentiment classifier trained on bag-of-words features.

Solution.

- $D = \{(x_i, y_i)\}_{i=1}^N$ where $x_i \in \mathbb{R}^V$ is a count vector over a vocabulary of size V , and $y_i \in \{0, 1\}$.
- $\mathcal{F} = \{f_\theta(x) = \sigma(\theta^\top x) : \theta \in \mathbb{R}^V\}$ where σ is the logistic function.
- $\mathcal{L}(\hat{y}, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$ (binary cross-entropy).
- $\mathcal{O}$ = Stochastic Gradient Descent (or Adam) on the average of $\mathcal{L}$ over D .

This single example contains every ingredient that scales, in essence, to a billion-parameter Transformer trained on the Web.

Example 3 (Pretext-task design — Hard). Suppose you wish to train a self-supervised model that learns useful representations of *Tunisian Arabic* text but you have no labels and no parallel translation data. Propose a concrete pretext task, identify the corresponding $(D, \mathcal{F}, \mathcal{L}, \mathcal{O})$, and discuss one risk specific to dialectal Arabic.

Solution sketch.

- **Pretext task.** Masked-token prediction: replace $\sim 15\%$ of tokens by a special [MASK], train the model to recover them.
- **Data D :** a corpus of Tunisian Arabic from social media, news, and forums (e.g. scraped Tunisian dialect from forums and Twitter).
- **Model $\mathcal{F}$:** a Transformer encoder with sub-word tokenization (BPE).
- **Loss $\mathcal{L}$:** cross-entropy on the masked positions.
- **Optimizer $\mathcal{O}$:** AdamW with warm-up.
- **Risk specific to dialectal Arabic.** Orthographic instability: the same word may be spelled in several ways (Latin script “arabizi”, native script with or without diacritics, code-switched fragments). The tokenizer must be trained on the actual data — a tokenizer pre-trained on Modern Standard Arabic will fragment dialect words badly.

1.3.8 Python snippet: train your first text classifier in 10 lines

The library `scikit-learn` provides a complete supervised pipeline.

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.linear_model import LogisticRegression
3 from sklearn.pipeline import Pipeline
4 from sklearn.metrics import classification_report
5
6 # Tiny illustrative dataset (replace with your own)
7 texts = ["I loved the food and the staff",      # positive
8          "The room was dirty and noisy",        # negative
9          "Fantastic value, will return",         # positive
10         "Worst stay of my life, never again"]   # negative
11 labels = [1, 0, 1, 0]
12
13 clf = Pipeline([
14     ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
15     ("logreg", LogisticRegression()),
16 ])
17 clf.fit(texts, labels)
18 print(clf.predict(["The breakfast was great", "Awful service"]))
```

This snippet instantiates each of the four ingredients: *data* (`texts`, `labels`), *model family* (TF-IDF + logistic regression), *loss* (the default cross-entropy of `LogisticRegression`), and *optimizer* (the default L-BFGS solver). The same recipe scales — with much larger data, deeper models, and different losses — to state-of-the-art systems.

1.3.9 Exercises

1. Place each method in its sub-discipline ($DL \subset ML \subset AI$): expert systems, decision trees, BERT, K-means clustering, A* search, GPT-4.
2. For each task, identify the most appropriate learning paradigm:
 - (a) Building a chatbot with 500 examples of (question, answer) pairs;
 - (b) Discovering five product-feature themes in 10,000 unlabelled customer reviews;
 - (c) Pretraining a French–Arabic encoder on a Web crawl with no aligned data.
3. Specify the four ingredients ($D, \mathcal{F}, \mathcal{L}, \mathcal{O}$) for a NER tagger using a feed-forward neural network on word-level features.
4. Discuss: why does the empirical risk on the *training* set systematically underestimate the empirical risk on *unseen* data? What practical safeguards address this?
5. (Open-ended.) For a Tunisian-bank social-media monitoring use case, design a three-stage pipeline:
 - (a) self-supervised pretraining of an embedding model on bank-related tweets;
 - (b) supervised fine-tuning for sentiment classification using 1,000 hand-labelled tweets;
 - (c) unsupervised clustering of negative tweets to identify recurring complaint themes.

Identify the inputs, outputs, paradigm, and metric for each stage.

1.4 Discriminative vs. generative models

1.4.1 Motivation

A foundational distinction in machine learning splits models into two families: those that learn the *conditional distribution* $P(y \mid x)$ of a label given an input (*discriminative*), and those that learn the *joint* or *marginal* distribution $P(x, y)$ or $P(x)$ (*generative*). The distinction is not academic — it determines what data you need, what you can do at inference time, and how you should evaluate the system. Crucially, every modern Large Language Model is a generative model that has learnt to solve discriminative tasks *by being prompted*; understanding this paradigm shift requires fluency with both families.

1.4.2 Approach by problem: classify or generate?

Problem. Given a corpus of customer reviews, contrast two systems:

1. System A: predicts the sentiment label of a new review.
2. System B: writes a new, plausible review.

Which probability distribution must each system represent? Could either be repurposed for the other's task?

Solution sketch. System A only needs the conditional distribution $P(\text{sentiment} \mid \text{review})$. It does not need to know what reviews look like in detail; it only needs the *decision boundary* between positive and negative reviews. System B, in contrast, needs $P(\text{review})$ — a model of what an arbitrary review looks like. Repurposing: System B can in principle answer the discriminative question by computing $P(\text{review} \mid \text{positive})/P(\text{review} \mid \text{negative})$ via Bayes, but System A cannot generate reviews; the generative model is therefore strictly more expressive (and harder to learn).

1.4.3 Discriminative models

Definition 1.4.1 (Discriminative model). A **discriminative model** parameterises the conditional probability of the label given the input,

$$P_{\theta}(y \mid x), \quad \theta \in \Theta.$$

At inference, predict $\hat{y} = \arg \max_y P_{\theta}(y \mid x)$ (or sample from the conditional, in soft-prediction settings).

Examples in NLP.

- Logistic regression for spam detection.
- Conditional random fields for NER.
- Encoder-only Transformers (BERT, RoBERTa, DeBERTa) fine-tuned for classification, NER, or extractive QA.

Strengths. Direct optimization of the quantity of interest (the label probability); typically less data-hungry than generative models for a fixed task; tends to outperform comparable generative models on discriminative tasks.

Limitations. Cannot generate text; cannot detect anomalous inputs (since $P(x)$ is unknown); usually requires labelled data.

1.4.4 Generative models

Definition 1.4.2 (Generative model). A **generative model** parameterises a probability distribution over the input space (or jointly over input and label):

$$P_{\theta}(x) \quad \text{or} \quad P_{\theta}(x, y).$$

At inference, generative models can: (i) sample new x from P_{θ} ; (ii) score the likelihood of an observed x ; and (iii) via Bayes, derive a discriminative classifier $P_{\theta}(y | x) \propto P_{\theta}(x | y)P_{\theta}(y)$.

Examples in NLP.

- N-gram language models: $P(w_t | w_{t-n+1}, \dots, w_{t-1})$.
- Hidden Markov Models: a generative process for sequences.
- Decoder-only Transformers (GPT, LLaMA, Claude, Gemini): autoregressive generative language models, $P(w_t | w_1, \dots, w_{t-1})$, scaled to billions of parameters.
- Encoder-decoder Transformers (T5, BART): generative for sequence-to-sequence tasks (translation, summarization).

Strengths. Can generate; can score arbitrary text; can be re-purposed for many downstream tasks via *prompting*; admit unlabelled training data by self-supervision.

Limitations. Far more parameters and data are required; harder to evaluate; their conditional probabilities derived via Bayes are typically less calibrated than those of a directly trained discriminative model.

1.4.5 Comparison and the modern blur

Aspect	Discriminative	Generative
Distribution	$P(y x)$	$P(x)$ or $P(x, y)$
Training data	Labelled (x, y) pairs	Often unlabelled x
NLP archetypes	BERT, logistic regression, CRF	GPT, n-grams, T5
Can predict labels	Yes, directly	Yes, via Bayes
Can generate text	No	Yes
Sample efficiency	Higher for the target task	Lower per task; reusable across tasks
Modern role	Specialised, fine-tuned models	Foundation models, prompted

Remark 1.4.1 (The modern blur). Until 2018, the discriminative–generative split correlated tightly with the model architecture: one trained a separate model for each task. Modern Large Language Models have eroded this distinction. A decoder-only Transformer such as GPT-4 is a generative model in its training objective (next-token prediction) but is routinely *prompted* to solve discriminative tasks: “*Classify the following review as positive or negative: ...*”. The model produces the answer as a continuation, exploiting its general linguistic competence. This paradigm — a single generative model handling the full task taxonomy — explains the centrality of LLMs in contemporary NLP.

1.4.6 The Bayes bridge

Proposition 1.4.1 (From generative to discriminative). *Any generative model with $P_\theta(x | y)$ and a class prior $P_\theta(y)$ induces a discriminative classifier via Bayes’ rule:*

$$P_\theta(y | x) = \frac{P_\theta(x | y) P_\theta(y)}{\sum_{y'} P_\theta(x | y') P_\theta(y')}.$$

Proof. Direct application of the definition of conditional probability: $P(y | x) = P(x, y)/P(x)$, with $P(x)$ obtained by marginalisation $\sum_{y'} P(x, y')$. The class-conditional decomposition $P(x, y) = P(x | y)P(y)$ completes the derivation. $\square$

Remark 1.4.2. The bridge is one-directional: a discriminative model $P(y | x)$ does *not* suffice to recover $P(x)$, because the marginal is integrated over y in a way that loses information about specific x -shapes.

1.4.7 Worked examples

Example 1 (Identifying the family — Easy). Classify each model as discriminative, generative, or both:

1. Naive Bayes spam classifier;
2. BERT fine-tuned for NER;
3. GPT-4 used to generate marketing copy;
4. GPT-4 prompted to label movie reviews as positive/negative;
5. A Transformer trained to predict the next token of a code corpus.

Solution.

1. **Generative** — Naive Bayes models $P(x | y)P(y)$ explicitly, even though it is most often used for classification.
2. **Discriminative** — BERT fine-tuned for NER directly models $P(\text{tag-sequence} | \text{tokens})$.
3. **Generative.**
4. **Generative used discriminatively** — the underlying model is generative but it is being applied to a discriminative task via prompting.

5. Generative.

Example 2 (Bayes bridge in practice — Medium). A Naive Bayes spam classifier estimates from training data:

$$P(\text{spam}) = 0.2, \quad P(\text{ham}) = 0.8.$$

For a particular short message x , the model also estimates

$$P(x \mid \text{spam}) = 10^{-4}, \quad P(x \mid \text{ham}) = 5 \times 10^{-6}.$$

Compute $P(\text{spam} \mid x)$.

Solution.

$$P(\text{spam} \mid x) = \frac{0.2 \times 10^{-4}}{0.2 \times 10^{-4} + 0.8 \times 5 \times 10^{-6}} = \frac{2 \times 10^{-5}}{2 \times 10^{-5} + 4 \times 10^{-6}} = \frac{20}{24} \approx 0.833.$$

The message is classified as spam with $\approx 83\%$ confidence. Note that the prior $P(\text{spam}) = 0.2$ is much smaller than 0.833 — the class-conditional likelihood ratio shifts the posterior strongly.

Example 3 (Modern LLM as a discriminative substitute — Hard). You wish to build a sentiment analysis system for product reviews. You can choose between:

- (a) Fine-tuning a 110M-parameter BERT model on 5,000 labelled reviews;
- (b) Prompting a 100B-parameter generative LLM with the same 5,000 reviews as in-context examples.

Discuss three trade-offs.

Solution.

1. **Cost.** Option (a) requires GPU-hours for fine-tuning and a small inference-time footprint; option (b) requires zero training but expensive per-call inference (LLM API or large GPU).
2. **Sample efficiency / accuracy.** BERT fine-tuned on 5,000 examples typically attains higher accuracy on the target task than an LLM with 5,000 in-context demonstrations, because the discriminative architecture is directly optimised for the task. The LLM, however, requires no update to its parameters and can be redirected to a new task by editing the prompt.
3. **Maintenance and adaptation.** BERT must be re-fine-tuned each time the label set changes; the LLM accommodates new labels by prompt rewriting, at no parameter cost. In a fast-evolving product catalogue, this can dominate other considerations.

1.4.8 Python snippet: probabilistic interpretation

```

1 import numpy as np
2 from sklearn.naive_bayes import MultinomialNB
3 from sklearn.linear_model import LogisticRegression
4 from sklearn.feature_extraction.text import CountVectorizer
5
6 texts = [
7     "great service highly recommend",      # positive
8     "terrible food never again",           # negative
9     "loved the room and the staff",        # positive
10    "worst experience of my life",          # negative
11 ]
12 labels = [1, 0, 1, 0]
13
14 X = CountVectorizer().fit_transform(texts)
15
16 # Generative: Multinomial Naive Bayes
17 gen = MultinomialNB().fit(X, labels)
18 print("Naive Bayes log P(x|y) and P(y):")
19 print("    log P(y)    :", gen.class_log_prior_)
20 print("    log P(x|y) shape:", gen.feature_log_prob_.shape)
21
22 # Discriminative: Logistic Regression
23 disc = LogisticRegression().fit(X, labels)
24 print("\nLogistic Regression P(y|x) on a held-out sentence:")
25 heldout = CountVectorizer(vocabulary=
26     CountVectorizer().fit(texts).vocabulary_).fit_transform(
27     ["the food was perfect"])
28 print("    P(y|x)    :", disc.predict_proba(heldout))

```

The `class_log_prior_` and `feature_log_prob_` attributes of `MultinomialNB` are exactly the generative quantities $\log P(y)$ and $\log P(x | y)$; the `predict_proba` of `LogisticRegression` is the discriminative quantity $P(y | x)$.

1.4.9 Exercises

- Classify each model as discriminative, generative, or hybrid:
 - Hidden Markov Model for POS tagging;
 - Conditional Random Field for NER;
 - Variational Auto-Encoder for text;
 - Decoder-only Transformer used as a chatbot;
 - Encoder-only Transformer used as a sentiment classifier.
- Suppose a generative spam model gives $P(x | \text{spam}) = 2 \times 10^{-5}$, $P(x | \text{ham}) = 10^{-3}$, with priors $P(\text{spam}) = 0.3$, $P(\text{ham}) = 0.7$. Compute the posterior probability that x is spam.

3. Argue why a generative model used for classification can outperform a discriminative model when training data is scarce. (Hint: think about what the generative model can “learn from” beyond the labels.)
4. In the LLM-as-classifier paradigm, the model receives a prompt such as “*Classify: positive or negative? — This restaurant was excellent.*” Identify three sources of variance in the predicted label and propose a mitigation for each.
5. (Synthesis.) For a moderation system that must (a) classify a tweet as toxic / safe and (b) detect previously unseen forms of toxicity, argue why a hybrid discriminative + generative architecture could outperform either alone.

1.5 Historical evolution: from rules to large language models

1.5.1 Motivation

NLP has reinvented itself roughly once per decade. Each new era inherits the data and applications of its predecessor but discards the previous era’s modelling assumptions. Understanding this recurring cycle pays off in two ways: it explains *why* certain methods exist (each is the answer to a previous limitation), and it provides a forecasting heuristic — when a modern method has a clear failure mode, the next era’s method is likely to address exactly that mode. The pattern also highlights a single guiding principle: *at every step, hand-engineered structure has been replaced by learned representations from data, and the volume of data and parameters has grown by orders of magnitude.*

1.5.2 Approach by problem: how would you build a chatbot in 1966?

Problem. Without machine learning, neural networks, or even substantial computing power, how could you build a system that converses in natural language with a human user? Sketch the design Joseph Weizenbaum chose for ELIZA in 1966.

Solution sketch. Weizenbaum designed *pattern-matching templates* keyed on user input. A user message such as “*I am unhappy.*” triggers a script associated with the keyword *unhappy*, which produces a response such as “*Why are you unhappy?*”. ELIZA performs no understanding; it shuffles surface forms by string substitution. Yet several users believed they were conversing with a human therapist — the so-called ELIZA effect. The strengths and limits of this pattern were the seeds of the next era: the system was *linguistically brittle, knowledge-bound, and domain-specific*, but proved that *some* useful interaction is achievable with extremely simple machinery.

1.5.3 Era 1 — Rule-based systems (1950s–1980s)

Approach. Linguists hand-write rules. Knowledge bases enumerate facts. Formal grammars (context-free, context-sensitive) describe syntax. Algorithms parse, transform, or generate strings according to the rules.

Representative systems.

- *Georgetown–IBM experiment* (1954): a Russian–English machine translator built from ~ 250 rules and a 6-rule grammar. Demoed live to a delighted press; declared the imminent end of human translators.
- *ELIZA* (Weizenbaum, 1966): the pattern-matching “psychotherapist” described above.
- *SHRDLU* (Winograd, 1972): a system that conversed in English about a blocks world.
- Rule-based spell checkers, dictionary-based morphological analysers, regex-driven information extraction.

Limitations. Rules do not scale: every new domain requires new rules, every new language requires re-coding. Ambiguity is hard to capture in rule form (which reading does the rule choose?). Coverage is brittle: any input not anticipated by the rules fails silently. Maintenance costs grow super-linearly with the rule base.

1.5.4 Era 2 — Statistical NLP (1990s–2000s)

Approach. Replace hand-written rules by *statistical models* estimated from text corpora. Use probabilistic decision rules: among parse trees, output the most probable; among translations, output the most likely target sentence. The IBM Candide statistical machine translation system, developed in the late 1980s and early 1990s, set the tone.

N-gram language models. The era’s flagship is the n-gram. Given a word sequence $w_1 \dots w_T$, factorize:

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}) \approx \prod_{t=1}^T P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

The *Markov assumption* ($n-1$ words of context suffice) makes the model trivial to estimate by counting in a corpus:

$$\hat{P}(w_t \mid w_{t-1}, \dots, w_{t-n+1}) = \frac{\#(w_{t-n+1}, \dots, w_t)}{\#(w_{t-n+1}, \dots, w_{t-1})}.$$

Other emblematic methods.

- Hidden Markov Models for POS tagging and speech recognition.
- Bag-of-Words and TF–IDF for text classification and information retrieval.
- Maximum-entropy / log-linear models for sequence labelling.

Limitations. Sparse data: most n-grams of order ≥ 4 never appear in any corpus. Smoothing techniques (Good–Turing, Kneser–Ney) patch the worst sparsity but do not solve the underlying problem. The context window is hard-coded and small. There is no notion of semantic similarity — “*cat*” and “*kitten*” are as unrelated to the model as “*cat*” and “*democracy*”.

1.5.5 Era 3 — Neural networks and word embeddings (2013–2017)

Approach. Represent each word as a *dense vector* learnt from data. Words that appear in similar contexts receive similar vectors. The model thus generalises across surface forms.

Landmark contributions.

- *Word2Vec* (Mikolov et al., 2013): two shallow neural architectures (*skip-gram*, *CBOW*) train word vectors by predicting context from a centre word, or vice-versa. The vectors exhibit striking algebraic regularities: $\vec{\text{king}} - \vec{\text{man}} + \vec{\text{woman}} \approx \vec{\text{queen}}$.
- *GloVe* (Pennington et al., 2014): global matrix factorization of the word–context co-occurrence matrix.
- *FastText* (Bojanowski et al., 2017): extends Word2Vec to sub-word units, handling out-of-vocabulary words and morphologically rich languages.
- *Sequence models* (LSTMs, GRUs): recurrent networks process sequences without the fixed n-gram window, partially solving the long-context problem.
- *Encoder–decoder architectures with attention* (Bahdanau et al., 2015): a neural translator can “*attend*” to all source positions when generating each target token.

Limitations.

- Word2Vec / GloVe / FastText produce *static* embeddings: “*bank*” has one vector, regardless of whether it refers to a financial institution or a riverside.
- Recurrent networks process sequences strictly left-to-right (or right-to-left), making long-range dependencies hard to capture and parallelisation difficult.

1.5.6 Era 4 — Transformers and large language models (2017–present)

Approach. A new architecture — the *Transformer* — replaces recurrence by *self-attention*, allowing every token to directly attend to every other token in the sequence. The Transformer decouples sequence length from depth and is fully parallelisable, enabling training on data and parameter scales unattainable for RNNs.

Three foundational papers.

- Vaswani et al. (2017), “*Attention Is All You Need*”: introduces the Transformer.
- Devlin et al. (2018), *BERT*: encoder-only Transformer pretrained by masked language modelling, then fine-tuned on downstream classification, NER, QA. State-of-the-art on dozens of benchmarks.
- Radford et al. (2018, 2019, 2020), *GPT-1*, *GPT-2*, *GPT-3*: decoder-only Transformer pretrained as a causal language model. Scaling demonstrated *emergent abilities* — in-context learning, instruction following, chain-of-thought reasoning — not present in smaller models.

Hallmarks of the era.

- **Contextual embeddings.** A token’s representation depends on its surrounding context: “*bank*” in a financial sentence and in a river sentence receive different vectors.
- **Self-supervised pretraining at Web scale.** Models trained on hundreds of billions of tokens of unlabelled text.
- **Foundation models.** A single pretrained model is adapted (by fine-tuning, prompting, or retrieval augmentation) to dozens of downstream tasks.
- **Scaling laws** (Kaplan et al., 2020; Hoffmann et al., 2022): test loss decays predictably as a power law in parameters, data, and compute, providing a quantitative roadmap for model design.
- **Reasoning and tool use.** Modern LLMs can be prompted to produce intermediate reasoning steps (*chain-of-thought*), to call external tools, or to be aligned with human preferences via reinforcement learning from human feedback (RLHF).

1.5.7 Synthetic comparison

Era	Representation	Semantics	Scalability	Period
Rules	Symbolic	Manual	Low	1950–19
Statistical (BoW, n-grams)	Counts / sparse vectors	Surface only	Medium	1990–20
TF–IDF	Weighted counts	Partial	Medium	1990–20
Word2Vec / GloVe / FastText	Static dense vectors	Yes (static)	High	2013–20
Transformers / LLMs	Contextual dense vectors	Yes (contextual)	Very high	2017–pr

1.5.8 The recurring pattern

Remark 1.5.1. Each era addressed a specific failure of its predecessor:

- Rules → statistical: *rules don’t scale or handle ambiguity probabilistically.*
- Statistical → neural: *counts cannot capture semantic similarity.*
- Static embeddings → Transformers: *a word’s meaning depends on context.*

A productive question to ask of any present-day NLP system is: *what is its specific failure mode, and what kind of method could fix it?* The most likely answer points at the next era.

1.5.9 Worked examples

Example 1 (Era identification — Easy). Place each method in its era:

1. Pattern-matching chatbot;
2. Trigram language model with Kneser–Ney smoothing;
3. BERT fine-tuned for sentiment;

4. Word2Vec skip-gram embeddings;
5. Hand-coded regex-based information extractor.

Solution. (1) Rule-based; (2) Statistical; (3) Transformers / LLMs; (4) Neural / static embeddings; (5) Rule-based.

Example 2 (Identifying era from a failure mode — Medium). A deployed translation system from Modern Standard Arabic to French is fluent on news articles but produces incoherent translations on Tunisian dialect tweets. Which era’s failure mode is this, and what era of method would address it?

Solution. The failure is a *distributional mismatch*: the training data does not cover the target dialect’s surface forms, so the model fails to generalise. This is most consistent with a *static-embedding-era* (or earlier) limitation, because contextual Transformer-era models trained on dialect data would, in principle, handle the variability through sub-word tokenisation and contextualisation. The fix is therefore to (a) introduce sub-word tokenisation (FastText-/BPE-style) and (b) pretrain on dialect data — both Transformer-era techniques.

Example 3 (The bigram inheritance — Hard). Show that in the limit where the context window is reduced to one previous token, a Transformer language model degenerates into a bigram model.

Solution sketch. A Transformer language model defines $P(w_t \mid w_1, \dots, w_{t-1})$ via attention over the entire history. If we restrict the model so that each token attends only to its immediate predecessor (a *causal mask of width 1*), the conditional becomes $P(w_t \mid w_{t-1})$. With sufficient capacity to memorise the bigram counts, the model would, at convergence, recover the same conditional probabilities as a bigram language model trained on the same corpus. This thought experiment shows that statistical and modern approaches lie on a spectrum of context-window choices; modern systems differ primarily in the *breadth and learned weighting* of context, not in their underlying probabilistic objective.

1.5.10 Python snippet: a four-line bigram language model

The simplest statistical-era artefact is illuminating to implement and evaluate.

```

1 from collections import Counter
2 import math, random
3
4 text = ("the_cat_sat_on_the_mat_the_cat_sat_on_the_floor_
5         "the_dog_sat_on_the_mat_the_dog_ran_on_the_floor").split()
6
7 bigrams = Counter(zip(text, text[1:]))
8 unigrams = Counter(text)
9
10 def P(w_next, w_prev):
11     """Conditional bigram probability with smoothing."""
12     return (bigrams[(w_prev, w_next)] + 1) / (unigrams[w_prev] +
13         len(unigrams))
14
15 def generate(start, n):
16     seq = [start]
17     for _ in range(n):

```



```

17     candidates = [w for w in unigrams if (seq[-1], w) in
18                    bigrams]
19     if not candidates:
20         break
21     weights = [P(w, seq[-1]) for w in candidates]
22     seq.append(random.choices(candidates, weights=weights)[0])
23     return "␣".join(seq)
24 print(generate("the", 8))

```

The same conceptual machinery — conditional probabilities, sampling — powers GPT-4, scaled by ten orders of magnitude.

1.5.11 Exercises

1. Date each method to its era (rules / statistical / static neural / Transformer):
 - (a) Hidden Markov Model for speech recognition;
 - (b) Convolutional neural network for sentiment classification;
 - (c) Mistral 7B large language model;
 - (d) Porter stemming algorithm;
 - (e) TF-IDF cosine retrieval.
2. Why did n-gram models choose $n \leq 5$ in practice, despite the formal benefit of larger n ? Identify the precise statistical phenomenon and the smoothing techniques designed to mitigate it.
3. Word2Vec “solves” the n-gram era’s most visible failure (no semantic similarity). What new failure does it introduce, and which era addresses it?
4. Discuss: each era replaces *hand engineering* by *data-driven learning*. What aspect of NLP, in your judgement, is still hand-engineered today, and what kind of method might learn it?
5. (Open-ended forecasting.) Identify a clear failure mode of contemporary LLMs (e.g. hallucination, lack of calibrated uncertainty, poor multilingual coverage on low-resource languages such as Tunisian dialects). Speculate on the methodological direction the next era might take to address it.

Chapter 2

Text Representation and Evaluation

A computer cannot operate on raw text. Before any model can “read” a sentence, the sentence must be *tokenized* (cut into discrete units), *numericalized* (converted into integer indices), and *vectorized* (embedded in a vector space). This chapter develops the chain rigorously: tokenization granularities, sparse and dense vector-space models, semantic similarity, and the evaluation metrics by which we compare NLP systems.

2.1 Tokenization fundamentals

2.1.1 Motivation

Tokenization is the unsung hero of every NLP pipeline. A poor tokenizer silently undermines the most sophisticated downstream model: out-of-vocabulary words become unknown tokens, morphological variants are treated as unrelated, multi-word expressions are fragmented. Conversely, a tokenizer aligned with the data distribution can shrink vocabulary size, accelerate training, and improve generalisation. Modern Large Language Models all rely on a single technical choice — *sub-word tokenization* — whose strengths and limitations every practitioner should understand.

2.1.2 Approach by problem: how many tokens are in this sentence?

Problem. Consider the sentence

“I’ve been to Sfax-University; it’s amazing!”

Count how many “tokens” it contains under each of the following naive policies:

1. split on whitespace;
2. split on whitespace and punctuation;
3. split into individual characters.

Solution sketch.

1. Whitespace only: {I’ve, been, to, Sfax-University;, it’s, amazing!} — 6 tokens. Punctuation glued to words; “I’ve” is a single token.

2. Whitespace and punctuation: {I, 've, been, to, Sfax, -, University, ;, it, 's, amazing, !} — 12 tokens. The contraction *I've* is split, but so is the legitimate compound *Sfax-University*.
3. Characters: 35 tokens. Punctuation, capitalisation and spaces are all preserved, but every word is fragmented; the model would have to relearn “what is a word” from data.

Each policy entails a trade-off between vocabulary size and information preserved. There is no universally correct answer; the choice is engineering, guided by the downstream task and the language being modelled.

2.1.3 Definition and the tokenization pipeline

Definition 2.1.1 (Tokenization). **Tokenization** is the process of converting a raw text string into an ordered sequence of discrete units called **tokens**, drawn from a finite inventory called the **vocabulary** V .

Definition 2.1.2 (Numericalization). **Numericalization** is the bijective mapping $V \leftrightarrow \{0, 1, \dots, |V| - 1\}$ that assigns each token an integer identifier.

The full pipeline is: *raw text* $\rightarrow$ tokens $\rightarrow$ token IDs $\rightarrow$ tensor inputs to a model. Every NLP system is built on top of these three trivially simple but practically critical steps.

2.1.4 Three levels of granularity

Level	Vocabulary size	Sequence length	OOV behaviour
Word	Very large (10^5 – 10^6)	Short	Unknown words become [UNK]
Character	Tiny (~ 100)	Very long	No OOV (any character is in V)
Sub-word	Moderate (10^4 – 10^5)	Moderate	Robust: any word splits into known pieces

Word tokenization. The intuitive default. Tokens are entire words separated by whitespace and punctuation rules. Strengths: each token carries lexical meaning, and downstream models converge faster on a small annotated dataset. Weaknesses: vocabulary explodes for morphologically rich languages (*Arabic*, *Turkish*, *Finnish*); typos and rare words become [UNK]; cannot generalise across morphological variants (*run* / *runs* / *running* are three unrelated tokens).

Character tokenization. Each character is a token. Strengths: vocabulary is bounded and stable across languages; no out-of-vocabulary problem (any unseen word becomes a sequence of known characters). Weaknesses: sequences become 5 – $10\times$ longer, increasing both memory and compute; the model must learn from scratch which character sequences form meaningful units.

Sub-word tokenization. The dominant choice in 2020s NLP. Vocabulary contains *frequent words as wholes* and *rare words split into meaningful pieces*. The split is learnt from data using algorithms such as Byte-Pair Encoding (BPE), WordPiece (BERT) and SentencePiece (XLM-R, T5). Sub-word tokenization combines the strengths of word-level (semantic units for common words) and character-level (no OOV) approaches.

2.1.5 Pre-processing operations

Tokenization is often combined with ad-hoc transformations that modify or normalise the input string before splitting.

- **Lowercasing:** “Apple” and “apple” become equivalent. Useful for spelling-insensitive tasks but destroys signal in NER (a capitalised *Apple* is more likely the company).
- **Punctuation removal:** simplifies vocabulary but loses sentence boundaries.
- **Stop-word removal:** discard high-frequency words (*the, of, in*) that carry little task-specific information. Useful for keyword search; harmful for syntax-sensitive tasks.
- **Stemming:** chop morphological suffixes by hand-coded rules (Porter, Snowball). *running* → *run*. Fast but linguistically crude.
- **Lemmatization:** map a word to its dictionary form using a morphological analyser. *better* → *good*. Slower but linguistically principled.

Remark 2.1.1. Modern large language models perform little or no manual pre-processing: they are trained on raw text with sub-word tokenization, and the model itself learns whatever normalisations it needs. Heavy pre-processing is typical of statistical-era pipelines and remains useful for small-data, classical-ML systems.

2.1.6 Byte-Pair Encoding (BPE) in detail

BPE is the simplest sub-word algorithm and the workhorse of GPT-2, GPT-3, GPT-4, RoBERTa, and many others. It iteratively merges the most frequent pair of adjacent symbols.

Algorithm.

1. Start with a vocabulary that contains every individual character of the corpus.
2. Treat each word as a sequence of characters with an end-of-word marker.
3. Count the frequency of every adjacent symbol pair across the corpus.
4. Merge the most frequent pair into a new symbol; add it to the vocabulary.
5. Repeat steps 3–4 until a target vocabulary size is reached.

Example 2.1.1 (A miniature BPE run). Suppose the corpus consists of the words `low`, `lower`, `newest`, `widest`, with frequencies 5, 2, 6, 3. The initial vocabulary is {`l`, `o`, `w`, `e`, `r`, `n`, `s`, `t`, `i`, `d`, `</w>`}. Counting pairs:

$$e s = 6 + 3 = 9, \quad s t = 9, \quad l o = 7, \quad o w = 7, \dots$$

The pair `e s` (9 occurrences) is merged first into a new symbol `es`. Re-count, merge again — now `es t` (9 occurrences) into `est`. Continue: `lo`, `low`, `ne`, `new`, ... After enough merges, frequent words like `low</w>` become single tokens, while rare words remain decomposed. A new word like `lowest` is then tokenized as `low` + `est`, even though it never appeared in training.

Why BPE works. The algorithm provides a natural rate–distortion trade-off. As vocabulary grows, the average sequence length shrinks, but the “unit cost” per token increases (each token carries more information). Empirically, vocabularies of 30k–100k sub-words give an excellent trade-off across many languages.

2.1.7 Multilingual and dialectal challenges

Tokenization choices that are adequate for English may fail for other languages.

- **Whitespace-free scripts.** Chinese, Japanese, Thai write without spaces. Word tokenization requires segmentation algorithms (e.g. Jieba, MeCab).
- **Morphologically rich languages.** Arabic and Turkish concatenate prefixes and suffixes onto a stem, generating very large surface-form inventories. Sub-word tokenization is essential.
- **Code-switching and dialects.** Tunisian Arabic frequently mixes Modern Standard Arabic, French, and Latin-script transliterations (“arabizi”). A general-purpose multilingual tokenizer (e.g. XLM-R’s SentencePiece) is recommended over single-language tokenizers.
- **Right-to-left scripts.** Display order and logical order can differ; numerical IDs operate on logical order, but evaluation tools must be aware of the visual rendering.

2.1.8 Worked examples

Example 1 (Granularity choice — Easy). You are training a classifier on 1000 short SMS messages, mostly in English with frequent typos. Which tokenization granularity is most appropriate, and why?

Solution. Sub-word tokenization. Word-level would suffer from OOV (typos produce never-seen surface forms); character-level would lengthen sequences and dilute the signal in a small dataset. Sub-word splits typos into recognised pieces (happy → ha + py) without exploding the vocabulary.

Example 2 (BPE iteration — Medium). Apply two iterations of BPE to the toy corpus {aaab, aabb, abab} (each of frequency 1). Show the resulting vocabulary after each merge.

Solution. Initial vocabulary: {a, b}.

Iteration 1: count pairs — a a appears in aaab (twice) and aabb (once) and abab (zero) = 3; a b appears in aaab (once) and aabb (once) and abab (twice) = 4; b a appears once (in abab); b b once. Most frequent: a b (4). Merge into ab. Vocabulary: {a, b, ab}. Re-tokenization: aaab → a a ab; aabb → a ab b; abab → ab ab.

Iteration 2: count new pairs — a a (in aaab, once) = 1; a ab (in aaab and aabb) = 2; ab b (in aabb) = 1; ab ab (in abab) = 1. Most frequent: a ab (2). Merge into aab. Vocabulary: {a, b, ab, aab}.

Example 3 (Tokenizer mismatch — Hard). A pretrained English BERT tokenizer applied to Tunisian Arabic in Latin script (“arabizi”) splits the word n7ebek (“I love you”) as [n, ##7, ##e, ##bek]. Discuss the consequences for a downstream classifier and propose two remediation strategies.

Solution.

- **Consequences.** The numeric digit 7 (which substitutes for the Arabic letter ‘) is treated as foreign; the model sees an unfamiliar pattern with no semantic anchoring. The classifier’s representation of the word will be near-random, and its predictions for sentences containing arabizi will be poor.
- **Remediations.**
 1. Use a multilingual tokenizer (e.g. xlm-roberta-base’s SentencePiece) trained on much broader data including Latin-script Arabic.
 2. Train a domain-specific tokenizer on a Tunisian-Arabic corpus before fine-tuning the model. The merges will then capture frequent dialectal sub-words natively.

2.1.9 Python snippet: comparing tokenizers

```

1 # pip install nltk transformers
2 import nltk
3 from transformers import AutoTokenizer
4
5 text = "I've been to Sfax-University; it's amazing!"
6
7 # Word-level (NLTK)
8 nltk.download('punkt_tab', quiet=True)
9 print("Word tokens (NLTK):", nltk.word_tokenize(text))
10
11 # Sub-word (BERT WordPiece)
12 bert = AutoTokenizer.from_pretrained("bert-base-uncased")
13 print("BERT sub-word tokens:", bert.tokenize(text))
14
15 # Sub-word (GPT-2 byte-level BPE)
16 gpt2 = AutoTokenizer.from_pretrained("gpt2")
17 print("GPT-2 BPE tokens:", gpt2.tokenize(text))

```

The same input produces different tokenisations under different algorithms. Reading the tokenized output is the first debugging step when a model behaves unexpectedly.

2.1.10 Exercises

1. For each scenario, recommend a tokenization granularity (word / character / sub-word) and justify briefly:
 - (a) Building a spelling corrector for French SMS;
 - (b) Training a sentiment classifier on 50 million Web reviews;
 - (c) Building a name-entity recognizer for German legal documents.
2. Explain in one paragraph why character-level tokenization, despite its $|V| \approx 100$, has *not* replaced sub-word tokenization in modern LLMs.
3. Tokenize the sentence “*The CEOs’ meeting was rescheduled.*” with two different policies (whitespace+punctuation, and a sub-word tokenizer of your choice). Comment on the differences for the words *CEOs’* and *rescheduled*.

4. Carry out three iterations of BPE on the toy corpus $\{\mathbf{xay}, \mathbf{xay}, \mathbf{yax}, \mathbf{xy}\}$ where each word has frequency 1. List the new vocabulary at each step.
5. (Synthesis.) For a multilingual Tunisian dialect dataset (mixing Arabic script, Latin-script arabizi, and French), discuss the trade-offs between (i) using a single multilingual sub-word tokenizer (e.g. XLM-R), (ii) training a domain-specific tokenizer on the dataset, and (iii) using language-specific tokenizers for each script and merging the outputs.

2.2 Sparse representations: Bag-of-Words and TF–IDF

2.2.1 Motivation

After tokenization, a text is a sequence of integer IDs. To feed such a sequence to a classical machine-learning algorithm — logistic regression, k -nearest-neighbours, support-vector machine — we need a *fixed-size vector* representation that does not depend on document length. The simplest such representation is the **Bag-of-Words** (BoW), which counts how often each vocabulary item appears. BoW and its weighted refinement **TF–IDF** dominated information retrieval and text classification for two decades; they remain strong baselines, fast to compute, easy to debug, and competitive on small or domain-specific data sets.

2.2.2 Approach by problem: are these two reviews similar?

Problem. Compare the following two hotel reviews:

- d_1 : “*The room was clean. The room was quiet.*”
- d_2 : “*The breakfast was great. The room was clean.*”

Propose a numerical representation for each that lets us measure their similarity, ignoring word order.

Solution sketch. Build a vocabulary by collecting all distinct words: $V = \{\text{the, room, was, clean, quiet, breakfast, great}\}$ (size 7). For each document, count how often each vocabulary word appears:

$$\mathbf{x}_{d_1} = (2, 2, 2, 1, 1, 0, 0)^\top, \quad \mathbf{x}_{d_2} = (2, 1, 2, 1, 0, 1, 1)^\top.$$

The two vectors share many components (e.g. both contain “the”, “room”, “was”, “clean”) and differ on others (*quiet* only in d_1 , *breakfast* / *great* only in d_2). Their dot product gives a coarse measure of similarity. This embarrassingly simple recipe is the starting point of all classical NLP.

2.2.3 Vector space models

Definition 2.2.1 (Vector space model). A **vector space model** (VSM) of a corpus is an embedding $\phi : \mathcal{T} \rightarrow \mathbb{R}^d$, where $\mathcal{T}$ is the set of textual objects (words or documents) and d is the embedding dimension. Documents are represented as vectors so that geometric operations (dot product, cosine similarity, distance) translate into linguistic operations (similarity, retrieval, clustering).

2.2.4 Bag-of-Words

Definition 2.2.2 (Bag-of-Words representation). Fix a vocabulary $V = \{w_1, w_2, \dots, w_{|V|}\}$. The **Bag-of-Words** (BoW) representation of a document d is the vector

$$\phi_{\text{BoW}}(d) = (\text{tf}(w_1, d), \text{tf}(w_2, d), \dots, \text{tf}(w_{|V|}, d)) \in \mathbb{R}^{|V|},$$

where $\text{tf}(w, d)$ is the *term frequency* (raw count) of word w in d . The representation is named “bag” because the order of words is discarded: the document is treated as a multiset.

Alternative weightings of tf.

- **Boolean:** $\text{tf}(w, d) \in \{0, 1\}$, indicating presence.
- **Raw count:** $\text{tf}(w, d) = \text{number of occurrences}$.
- **Log-normalized:** $\text{tf}(w, d) = \log(1 + \#(w, d))$, mitigating the bias toward long documents.
- **Length-normalized:** divide raw counts by document length so that all documents have ℓ_1 -norm 1.

2.2.5 The document–term matrix

Definition 2.2.3 (Document–term matrix). Given a corpus $\{d_1, \dots, d_N\}$ and a vocabulary V , the **document–term matrix** is the matrix $X \in \mathbb{R}^{N \times |V|}$ whose entry $X_{ij} = \text{tf}(w_j, d_i)$.

In practice $|V|$ is in the tens or hundreds of thousands, while only a few dozen distinct words appear in each document. The matrix is therefore extremely *sparse*: a vast majority of entries are zero. Efficient storage formats (compressed sparse row, CSR; compressed sparse column, CSC) preserve memory by storing only non-zero entries.

2.2.6 Limitations of Bag-of-Words

- **No word order:** the documents “*Tunisia beat Egypt*” and “*Egypt beat Tunisia*” yield identical BoW vectors despite their opposite meanings.
- **Equal weight to all words:** a function word like “*the*” contributes the same magnitude as a domain-specific term like “*Paracetamol*”, even though the latter is far more discriminative.
- **No semantic similarity:** “*cat*” and “*kitten*” are orthogonal vectors in $\mathbb{R}^{|V|}$ — as far apart as “*cat*” and “*democracy*”.
- **Curse of dimensionality:** $|V| \approx 50,000$ produces vectors that are computationally cheap (sparse) but statistically noisy — many features contribute marginally to any specific task.

The first two limitations are tackled in this section by TF-IDF; the third requires dense embeddings (Section 2.3). The first — the loss of order — is fundamentally unfixable in a bag-of-words framework and is a key driver of the move to recurrent and Transformer models.

2.2.7 TF-IDF

The intuition is to *down-weight* words that appear in many documents (because they discriminate poorly between documents) and *up-weight* words that appear only in a few (because their presence is informative).

Definition 2.2.4 (Inverse document frequency). Given a corpus of N documents, the **document frequency** of a word w is the number of documents that contain w at least once,

$$\text{df}(w) = |\{d \in \text{corpus} : w \in d\}|.$$

The **inverse document frequency** is

$$\text{idf}(w) = \log \frac{N}{\text{df}(w)}.$$

Common variants smooth the formula to avoid division by zero and dampen extreme values:

$$\text{idf}_{\text{smooth}}(w) = \log \left(\frac{N + 1}{\text{df}(w) + 1} \right) + 1.$$

Definition 2.2.5 (TF-IDF). The **TF-IDF** weight of word w in document d is the product

$$\text{tfidf}(w, d) = \text{tf}(w, d) \times \text{idf}(w).$$

Replacing each entry of the document-term matrix by the corresponding TF-IDF gives the TF-IDF document-term matrix.

Intuition.

- Words appearing in nearly every document have $\text{df}(w) \approx N$, hence $\text{idf}(w) \approx \log 1 = 0$. They are zeroed out — the same effect as stop-word removal, but data-driven.
- Words appearing in few documents have $\text{df}(w) \ll N$, hence large $\text{idf}(w)$. They retain (and amplify) their term-frequency contribution.
- Within a single document, TF-IDF still scales with raw term frequency, preserving emphasis on repeated key terms.

Property 2.2.1 (Boundedness of idf). For a corpus of N documents, $0 \leq \text{idf}(w) \leq \log N$ when idf uses the unsmoothed formula. The maximum is reached when w appears in exactly one document, the minimum when w appears in all of them.

2.2.8 TF-IDF as a probabilistic measure

Remark 2.2.1 (Information-theoretic flavour). $\text{idf}(w) = -\log \frac{\text{df}(w)}{N}$ is the negative log-probability that a randomly chosen document contains the word w . A rare word “surprises” us when it appears, contributing many bits of information; a common word contributes few. This is a discrete analogue of pointwise mutual information, and the connection has been formalised: TF-IDF approximates a maximum-likelihood estimate under specific generative assumptions.

2.2.9 Worked examples

Example 1 (BoW from scratch — Easy). For the two-document corpus

- d_1 : “the cat sat on the mat”
- d_2 : “the cat ate the fish”

build the vocabulary, the document–term matrix with raw counts, and compute $\text{idf}(w)$ for each word.

Solution. $V = \{\text{the, cat, sat, on, mat, ate, fish}\}$.

	the	cat	sat	on	mat	ate	fish
d_1	2	1	1	1	1	0	0
d_2	2	1	0	0	0	1	1
df	2	2	1	1	1	1	1
idf	$\log 1 = 0$	0	$\log 2$	$\log 2$	$\log 2$	$\log 2$	$\log 2$

“the” and “cat” carry no IDF weight (they appear in every document), while *sat*, *on*, *mat*, *ate*, *fish* all carry weight $\log 2 \approx 0.69$.

Example 2 (Computing TF–IDF — Medium). A four-document corpus has $\text{df}(\textit{Tunisia}) = 1$ and $\text{df}(\textit{the}) = 4$. Document d_1 contains *Tunisia* once and *the* four times.

(a) Compute $\text{tfidf}(\textit{Tunisia}, d_1)$ and $\text{tfidf}(\textit{the}, d_1)$ using the unsmoothed formula.

(b) Comment on the result: which word’s contribution dominates the document representation?

Solution. $\text{idf}(\textit{Tunisia}) = \log(4/1) = \log 4 \approx 1.39$. $\text{idf}(\textit{the}) = \log(4/4) = 0$. $\text{tfidf}(\textit{Tunisia}, d_1) = 1 \times 1.39 = 1.39$. $\text{tfidf}(\textit{the}, d_1) = 4 \times 0 = 0$. The discriminative word *Tunisia* dominates the representation, even though it appears only once. The function word *the*, which appears four times, is silenced. This is exactly the desired behaviour: a query for *Tunisia* should retrieve d_1 , while a query for *the* should not.

Example 3 (Spam filtering with TF–IDF — Hard). Consider a spam-filtering corpus. The word *lottery* appears in 50 out of 5,000 training emails, all of which are spam. The word *meeting* appears in 1,200 emails, half spam, half legitimate. Argue, using TF–IDF and the discriminative power of each feature, why a logistic-regression classifier built on TF–IDF features will give a much larger weight to *lottery* than to *meeting*.

Solution. The IDF of *lottery* is $\log(5000/50) = \log 100 \approx 4.6$, while the IDF of *meeting* is $\log(5000/1200) \approx 1.4$. Already, after the IDF re-weighting, *lottery* contributes more to the representation than *meeting* per occurrence. Logistic regression then assigns weights to maximise log-likelihood; *lottery* is correlated with the spam label with probability 1, while *meeting* is correlated with the spam label with probability 0.5. The resulting weight on *lottery* reflects both the higher IDF amplification and the stronger label correlation. The classifier is therefore much more sensitive to the word *lottery* when scoring a new e-mail.

2.2.10 Python snippet: from text to TF–IDF in 5 lines

```
1 from sklearn.feature_extraction.text import CountVectorizer,
   TfIdfVectorizer
2
3 corpus = [
```

```

4         "the_room_was_clean_the_room_was_quiet",
5         "the_breakfast_was_great_the_room_was_clean",
6         "service_was_slow_but_the_rooms_were_clean",
7     ]
8
9     # Step 1 : raw counts
10    counts = CountVectorizer().fit_transform(corpus)
11    print("Raw counts shape:", counts.shape)
12    print("Raw counts dense:\n", counts.toarray())
13
14    # Step 2 : TF-IDF transform
15    tfidf = TfidfVectorizer().fit(corpus)
16    X = tfidf.transform(corpus)
17    print("\nVocabulary:", tfidf.get_feature_names_out())
18    print("TF-IDF matrix:\n", X.toarray().round(2))
19
20    # Inspect the IDF of each token
21    for term, score in zip(tfidf.get_feature_names_out(), tfidf.idf_):
22        print(f"_{term}_idf({term}) = {score:.3f}")

```

2.2.11 Exercises

1. Build the BoW vocabulary, the document-term matrix, the IDF vector and the full TF-IDF matrix (smoothed) for the corpus
 - d_1 : “NLP is fun and useful”
 - d_2 : “Tunisia is a country in North Africa”
 - d_3 : “NLP is widely used in Tunisia”
2. Show that for a corpus of N documents, the maximum possible IDF (unsmoothed) is $\log N$ and is reached when a word appears in exactly one document. Conversely, show that words appearing in every document have $\text{idf}(w) = 0$ and contribute zero TF-IDF weight.
3. Two documents, d_1 and d_2 , share exactly the same set of words but with different counts. Are their BoW vectors necessarily different? Are their normalised BoW vectors (length-normalised) necessarily different?
4. TF-IDF is purely lexical and shares no information across morphological variants (*run* and *running* are unrelated columns of the document-term matrix). Propose two engineering remedies (one classical, one modern) and one situation in which each is preferable.
5. (Open-ended.) Suppose you must build a search engine over a corpus of 100,000 Tunisian-Arabic news articles. Discuss whether you would prefer a TF-IDF index, a dense-vector index (Section 2.3), or a hybrid of the two. Identify a concrete query that one method handles but not the other.

2.3 Dense representations: Word2Vec, GloVe, FastText

2.3.1 Motivation

Sparse Bag-of-Words and TF-IDF representations encode lexical presence but not *meaning*. To a sparse model, the vectors of *cat* and *kitten* are orthogonal — as unrelated as *cat* and *democracy*. This severely limits generalisation: a classifier trained on documents containing *cat* cannot directly transfer its decision to documents containing *kitten*, even though the two words mean almost the same thing. *Dense word embeddings* solve this problem by mapping every word to a low-dimensional real-valued vector such that semantically similar words receive similar vectors. The idea, in a single line: *a word's meaning is determined by the company it keeps*.

2.3.2 Approach by problem: where does meaning live?

Problem. Given a corpus of one billion English sentences, how could you produce, without any human annotation, a 300-dimensional vector for each English word such that synonyms end up close together?

Solution sketch. Read the corpus and count, for every pair of words (w, c) , how often they co-occur within a small context window. Words that share many contexts (*cat* / *kitten* both appear near *purr*, *fur*, *meow*, *pet*) will have similar count profiles. Reduce these high-dimensional count vectors to a low-dimensional latent space (e.g. via matrix factorization), and similar words will project to nearby points. The full algorithm refines this idea, but the core insight is captured by a single principle from mid-20th-century linguistics.

2.3.3 The distributional hypothesis

Definition 2.3.1 (Distributional hypothesis, Harris 1954, Firth 1957). The semantic similarity of two words is well approximated by the similarity of their typical *contexts*. Formally, if $C(w)$ denotes the multiset of words appearing within a fixed-size window of w across a corpus, then

$$\text{sim}_{\text{semantic}}(w_1, w_2) \approx \text{sim}_{\text{distributional}}(C(w_1), C(w_2)).$$

This empirical hypothesis has held up remarkably well: every modern word-embedding algorithm is, at root, a way to operationalise it.

2.3.4 Word2Vec

Mikolov et al. (2013) introduced two shallow neural architectures that learn dense word vectors from raw text. Each word w is associated with two vectors: a *centre vector* $\mathbf{v}_w \in \mathbb{R}^d$ and a *context vector* $\mathbf{u}_w \in \mathbb{R}^d$, both stored in lookup tables and updated by gradient descent.

Skip-gram. Given a centre word, predict the surrounding context words. Formally, for a corpus tokenized as $w_1, w_2, \dots, w_T$ and a context window of size m ,

$$\mathcal{L}_{\text{skip-gram}} = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} \mid w_t),$$

with the conditional modelled as a softmax:

$$P(c \mid w) = \frac{\exp(\mathbf{u}_c^\top \mathbf{v}_w)}{\sum_{c' \in V} \exp(\mathbf{u}_{c'}^\top \mathbf{v}_w)}.$$

CBOW (Continuous Bag-of-Words). Symmetric: predict the centre word from the average of context vectors. Faster to train but slightly less accurate on rare words.

Negative sampling. Computing the softmax over the entire vocabulary is too expensive when $|V| \sim 10^6$. The trick is to replace it with a binary classification: for each true context pair (w, c) , draw k “noise” words $\tilde{c}_1, \dots, \tilde{c}_k$ from a unigram distribution and minimise

$$-\log \sigma(\mathbf{u}_c^\top \mathbf{v}_w) - \sum_{i=1}^k \log \sigma(-\mathbf{u}_{\tilde{c}_i}^\top \mathbf{v}_w).$$

This is a discriminative surrogate for the original generative loss; empirically, $k = 5\text{--}15$ is sufficient for high-quality embeddings.

2.3.5 GloVe

Pennington et al. (2014) take a complementary route: factor the global word–context co-occurrence matrix instead of training on local windows.

Setup. Let $X \in \mathbb{R}^{|V| \times |V|}$ be the matrix whose entry X_{ij} counts how often word j appears in the context of word i . The aim is to find vectors $\mathbf{v}_i, \mathbf{u}_j \in \mathbb{R}^d$ and biases b_i, c_j such that

$$\mathbf{v}_i^\top \mathbf{u}_j + b_i + c_j \approx \log X_{ij}.$$

The factorization minimises a weighted least-squares objective:

$$\mathcal{L}_{\text{GloVe}} = \sum_{i,j} f(X_{ij}) (\mathbf{v}_i^\top \mathbf{u}_j + b_i + c_j - \log X_{ij})^2,$$

where f is a weighting function that down-weights very rare and very frequent co-occurrences.

Skip-gram and GloVe are dual. Levy and Goldberg (2014) showed that skip-gram with negative sampling implicitly factors a shifted pointwise mutual information matrix,

$$\mathbf{v}_w^\top \mathbf{u}_c \approx \log \frac{P(w, c)}{P(w)P(c)} - \log k,$$

where k is the number of negative samples. The two algorithms thus solve closely related optimisation problems with different objectives and weightings; they typically yield comparable embedding quality.

2.3.6 FastText

Bojanowski et al. (2017) extend Word2Vec by embedding *sub-word units* rather than entire words. A word is represented as the sum of vectors of its character n -grams (with $n = 3, 4, 5, 6$):

$$\mathbf{v}_{\text{word}} = \sum_{g \in \mathcal{G}_{\text{word}}} \mathbf{v}_g.$$

Two consequences:

- **No more OOV.** Any unseen word can be embedded as a sum of its n -gram vectors — including misspellings, neologisms, and morphological variants.
- **Better for morphologically rich languages.** The vectors of *run*, *runs*, *running* share most of their n -grams, so the model captures morphological similarity automatically. This is especially valuable for Arabic, Turkish, Finnish, and dialectal varieties.

2.3.7 Vector arithmetic and the analogy task

A striking empirical property of dense embeddings is that they encode semantic relations *linearly*.

Property 2.3.1 (Word analogies). For high-quality embeddings, relational analogies of the form

$$A \text{ is to } B \text{ as } C \text{ is to } D$$

are often captured by

$$\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C \approx \mathbf{v}_D,$$

in the sense that the nearest neighbour of $\mathbf{v}_B - \mathbf{v}_A + \mathbf{v}_C$ in the embedding space (excluding A, B, C themselves) is D .

Examples.

- Gender: $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$.
- Country-capital: $\overrightarrow{\text{Tunis}} - \overrightarrow{\text{Tunisia}} + \overrightarrow{\text{France}} \approx \overrightarrow{\text{Paris}}$.
- Verb tense: $\overrightarrow{\text{walked}} - \overrightarrow{\text{walking}} + \overrightarrow{\text{running}} \approx \overrightarrow{\text{ran}}$.

The analogy task became the dominant intrinsic-evaluation benchmark for word embeddings throughout the mid-2010s.

2.3.8 Limitations: static, monolingual, biased

- **Static.** Each word receives one vector regardless of context. The polysemous word *bank* has a single embedding that mixes its financial and riverside senses — useful as an average, useless for disambiguation. Section 2.3.9 previews the contextual fix.
- **Monolingual.** Word2Vec / GloVe / FastText vectors are trained on a single language. Cross-lingual alignment requires extra mechanisms.

- **Biases inherited from the training corpus.** The data is the model. If the corpus encodes social biases, the embeddings inherit them: $\overrightarrow{\text{nurs}\bar{e}}$ may end up closer to $\overrightarrow{\text{wom}\bar{a}\text{n}}$ than to $\overrightarrow{\text{ma}\bar{n}}$, with measurable downstream consequences. A growing body of literature studies and mitigates these effects.

2.3.9 Toward contextual embeddings

Static embeddings give one vector per word type. Modern Transformer encoders (BERT, RoBERTa, ...) instead produce one vector per word *token in context*: the same word *bank* receives different vectors when it appears in “*the bank approved my loan*” and in “*we sat on the bank of the river*”. Contextualisation is the topic of Part 2 of this course; for now, treat static embeddings as the entry point and the conceptual ancestor.

2.3.10 Worked examples

Example 1 (Distributional similarity — Easy). In a small corpus, the words *cat* and *dog* both frequently co-occur with *pet*, *food*, *bowl*, while *democracy* co-occurs with *election*, *vote*, *government*. Justify intuitively why a word-embedding algorithm will place $\overrightarrow{\text{cat}}$ closer to $\overrightarrow{\text{dog}}$ than to $\overrightarrow{\text{democracy}}$.

Solution. The distributional hypothesis predicts that words with similar context distributions are placed close in embedding space. The algorithms operationalise this by training each word’s vector to be predictive of (or compatible with) its observed contexts. Since *cat* and *dog* have similar context distributions, their vectors will be similar; *democracy*’s contexts are nearly disjoint, so its vector will be far from both.

Example 2 (Skip-gram softmax explicit — Medium). A toy vocabulary contains only $\{\text{cat}, \text{dog}, \text{the}\}$, with embeddings $\mathbf{v}_{\text{cat}} = (1, 0)$, $\mathbf{v}_{\text{dog}} = (0.9, 0.1)$, $\mathbf{v}_{\text{the}} = (0, 1)$ and context vectors $\mathbf{u}_w = \mathbf{v}_w$ for simplicity. Compute $P(\text{dog} \mid \text{cat})$ under the skip-gram softmax.

Solution. The dot products are $\mathbf{u}_{\text{cat}}^\top \mathbf{v}_{\text{cat}} = 1$, $\mathbf{u}_{\text{dog}}^\top \mathbf{v}_{\text{cat}} = 0.9$, $\mathbf{u}_{\text{the}}^\top \mathbf{v}_{\text{cat}} = 0$.

$$P(\text{dog} \mid \text{cat}) = \frac{e^{0.9}}{e^1 + e^{0.9} + e^0} = \frac{2.46}{2.72 + 2.46 + 1} \approx \frac{2.46}{6.18} \approx 0.398.$$

The model assigns $\sim 40\%$ of its conditional mass to *dog* (close to but slightly less than *cat* itself, which would self-predict with $\sim 44\%$).

Example 3 (FastText for Tunisian Arabic — Hard). The Tunisian-Arabic verb *n7eb* (“I love”) is unlikely to appear in a generic Arabic corpus, but its sub-word components (*n*, *7*, *he*, *eb*, ...) appear frequently in dialectal text. Argue why FastText would handle this word more robustly than Word2Vec, and write the formula for its vector.

Solution. Word2Vec treats *n7eb* as a single, unseen token: at inference, the lookup is undefined and the model falls back to a default [UNK] vector. FastText instead computes

$$\mathbf{v}_{n7eb} = \sum_{g \in \mathcal{G}(n7eb)} \mathbf{v}_g,$$

where $\mathcal{G}(n7eb)$ is the set of character *n*-grams of *n7eb* (with sentinels): for $n = 3$, $\mathcal{G} = \{\langle \text{n7}, \text{n7e}, \text{7eb}, \text{eb} \rangle, \dots\}$. Each *n*-gram has been seen and embedded during training; their sum is a valid embedding even though the whole word is novel. The resulting vector

encodes morphological and orthographic similarity to other dialectal verb forms, making downstream tasks (sentiment, NER) significantly more robust.

2.3.11 Python snippet: train and explore Word2Vec

```
1 # pip install gensim
2 from gensim.models import Word2Vec
3
4 corpus = [
5     "the_cat_sat_on_the_mat".split(),
6     "a_kitten_sleeps_on_the_couch".split(),
7     "dogs_and_cats_are_common_pets".split(),
8     "Tunisia_is_a_country_in_north_africa".split(),
9     "Tunis_is_the_capital_of_Tunisia".split(),
10    "Paris_is_the_capital_of_France".split(),
11 ]
12
13 model = Word2Vec(
14     sentences=corpus, vector_size=50, window=2,
15     min_count=1, sg=1, negative=5, epochs=200,
16 )
17
18 print("vec(cat)=", model.wv["cat"][:5], "...")
19 print("Most similar to 'cat':")
20 for w, s in model.wv.most_similar("cat", topn=3):
21     print(f"_{w:<12s}_cos-sim={s:.3f}")
22
23 # Analogy: Tunis - Tunisia + France ~= Paris
24 analogy = model.wv.most_similar(
25     positive=["France", "Tunis"], negative=["Tunisia"], topn=3,
26 )
27 print("\nTunis_is_to_Tunisia_as_?_is_to_France:")
28 for w, s in analogy:
29     print(f"_{w:<12s}_cos-sim={s:.3f}")
```

The toy corpus is too small to deliver high-quality vectors, but the API is identical to a corpus of billions of tokens.

2.3.12 Exercises

1. Restate the distributional hypothesis in your own words and explain why it is an *empirical* (rather than mathematical) claim. Give one sentence for which the hypothesis arguably fails.
2. Skip-gram and CBOW are dual training objectives. For each, argue qualitatively whether you expect better embeddings on rare or on frequent words, and why.
3. Show that for a uniform vocabulary distribution, the $\log P(w, c)/P(w)P(c)$ in Levy–Goldberg’s analysis simplifies to $\log P(c | w) + \log |V|$. Comment on the consequence for the implicit objective of skip-gram.

4. Construct three analogy tests of your own — one geographic, one syntactic, and one social — and predict whether you expect a generic Word2Vec model trained on Wikipedia to solve them.
5. (Synthesis.) Discuss why FastText is a natural choice for a sentiment classifier on Tunisian-Arabic Twitter data, while Word2Vec is more suitable for a clean English Wikipedia corpus. Identify two distinct linguistic features of dialectal Arabic that drive this preference.

2.4 Semantic similarity measures

2.4.1 Motivation

Once texts have been embedded in a vector space (sparse or dense), the natural way to compare them is geometric. “How similar are these two documents?” becomes “How close are their vectors?”. The choice of similarity measure is far from neutral — different measures embody different assumptions about what should count as similar. Three measures dominate practice: **cosine similarity**, **Euclidean distance**, and the **dot product**. Understanding their properties is essential for building search engines, clustering algorithms, recommender systems, and the retrieval-augmented generation pattern that sits at the heart of modern Question-Answering.

2.4.2 Approach by problem: ranking by similarity

Problem. You have embedded 10,000 news articles as TF-IDF vectors. A user submits the short query “*Tunisia football*”. Rank the articles by their relevance to the query.

Solution sketch. Embed the query with the same vectorizer used for the corpus, obtaining a query vector $\mathbf{q} \in \mathbb{R}^{|V|}$. For each article vector $\mathbf{x}_i$, compute a similarity score and sort articles in decreasing order of score. Two issues arise:

- Documents have very different lengths, hence very different ℓ_2 -norms. A long article will produce a large dot product simply because its vector is large, not because it is more relevant. We need a measure invariant to magnitude.
- Articles in different topical zones should rank far apart even when they share many function words. The measure must give weight to direction, not magnitude.

Both observations point to *cosine similarity*: the cosine of the angle between query and document vectors.

2.4.3 Three classical measures

Let $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$ be two vectors.

Definition 2.4.1 (Cosine similarity).

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2} = \frac{\sum_{i=1}^d a_i b_i}{\sqrt{\sum_{i=1}^d a_i^2} \sqrt{\sum_{i=1}^d b_i^2}}.$$

The values lie in $[-1, +1]$ and equal $+1$ exactly when $\mathbf{b}$ is a positive scaling of $\mathbf{a}$.

Definition 2.4.2 (Dot product).

$$\mathbf{a}^\top \mathbf{b} = \sum_{i=1}^d a_i b_i.$$

Faster to compute (no division or square roots) and equal to $\cos(\mathbf{a}, \mathbf{b}) \cdot \|\mathbf{a}\|_2 \cdot \|\mathbf{b}\|_2$. Sensitive to the magnitudes of both vectors.

Definition 2.4.3 (Euclidean distance).

$$d_2(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_2 = \sqrt{\sum_{i=1}^d (a_i - b_i)^2}.$$

Always non-negative; zero iff $\mathbf{a} = \mathbf{b}$. Sensitive to both magnitude and direction.

Definition 2.4.4 (Manhattan (ℓ_1) distance).

$$d_1(\mathbf{a}, \mathbf{b}) = \|\mathbf{a} - \mathbf{b}\|_1 = \sum_{i=1}^d |a_i - b_i|.$$

Used less often in NLP, but practical in high-dimensional spaces and when robustness to outliers matters.

2.4.4 Properties of cosine similarity

Property 2.4.1 (Magnitude invariance). For any $\lambda > 0$, $\cos(\lambda \mathbf{a}, \mathbf{b}) = \cos(\mathbf{a}, \mathbf{b})$.

Proof. Direct from the definition: scaling $\mathbf{a}$ by $\lambda > 0$ multiplies both numerator and denominator by λ , leaving the ratio unchanged. $\square$

This single property is decisive in NLP: TF-IDF vectors of long documents are scaled-up copies of those of shorter documents on the same topic. Cosine similarity correctly considers them equivalent in topic.

Property 2.4.2 (Cosine and Euclidean for unit vectors). If $\|\mathbf{a}\|_2 = \|\mathbf{b}\|_2 = 1$, then

$$d_2(\mathbf{a}, \mathbf{b})^2 = 2 - 2 \cos(\mathbf{a}, \mathbf{b}).$$

Proof. $d_2(\mathbf{a}, \mathbf{b})^2 = \|\mathbf{a}\|^2 - 2\mathbf{a}^\top \mathbf{b} + \|\mathbf{b}\|^2 = 1 - 2\mathbf{a}^\top \mathbf{b} + 1 = 2(1 - \cos(\mathbf{a}, \mathbf{b}))$ since the vectors are unit-norm. $\square$

Remark 2.4.1. Property 2.4.2 means that, after ℓ_2 -normalisation, ranking by cosine and ranking by Euclidean distance are equivalent (one is a monotonic function of the other). Practical libraries exploit this: they store ℓ_2 -normalised vectors and compute dot products, which are mathematically equivalent to cosine but considerably faster.

Property 2.4.3 (Cosine similarity is not a metric). $1 - \cos(\mathbf{a}, \mathbf{b})$ does not satisfy the triangle inequality and is therefore not a metric on $\mathbb{R}^d$. (After ℓ_2 -normalisation, the angular distance $\arccos(\cos(\mathbf{a}, \mathbf{b}))$ is a metric on the unit sphere.)

2.4.5 When to use which

Measure	Use when	Avoid
Cosine	Comparing texts of varying length; TF-IDF; static embeddings	Magnit
Dot product	Speed matters; vectors already ℓ_2 -normalised	Magnit
Euclidean	Clustering (k -means in normalised space); coordinates with absolute meaning	Docum
Manhattan	High-dimensional vectors with outliers; robustness matters	Dense

Rule of thumb. For text classification or retrieval with TF-IDF or static embeddings, use cosine. For clustering with k -means, ℓ_2 -normalise first then use Euclidean (equivalent to spherical k -means).

2.4.6 Applications

Semantic search. Embed every document; embed each incoming query; return the top- k documents by cosine similarity. This pattern powers “related articles” widgets, web search re-ranking, and the *retrieval* step of *retrieval-augmented generation* (RAG) for LLMs.

Document deduplication. In a corpus scraped from the Web, near-duplicate documents abound. Cluster cosine-similar pairs above a threshold (e.g. 0.95) and merge them.

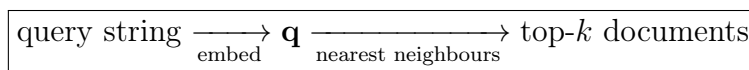
Recommendation. Represent users and items in the same vector space; recommend items closest to the user’s profile vector.

Clustering. Group documents into k topics by spherical k -means in the embedding space.

Cross-lingual similarity. If embeddings are aligned across languages, an Arabic document can be matched directly to its French paraphrase by cosine similarity, without translation.

2.4.7 The retrieval pattern

The pattern that re-appears throughout modern NLP, including in the inner loop of RAG systems, is:



At scale, exact nearest-neighbour search becomes infeasible. Approximate nearest-neighbour (ANN) algorithms — HNSW, FAISS, ScaNN, IVF-PQ — trade a small loss in precision for a 10–100× speed-up. We will use FAISS in the lab for a Tunisian-news search engine.

2.4.8 Worked examples

Example 1 (Cosine vs dot product on length — Easy). Two TF-IDF vectors on a vocabulary of $\{Tunisia, Tunis, beach, food\}$ are

$$\mathbf{a} = (1, 1, 0, 0)^\top, \quad \mathbf{b} = (3, 3, 0, 0)^\top.$$

Compute the cosine similarity and the dot product. Comment.

Solution.

$$\mathbf{a}^\top \mathbf{b} = 3 + 3 = 6, \quad \|\mathbf{a}\| = \sqrt{2}, \quad \|\mathbf{b}\| = \sqrt{18} = 3\sqrt{2}.$$

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{6}{\sqrt{2} \cdot 3\sqrt{2}} = \frac{6}{6} = 1.$$

The dot product is 6, but cosine is 1 — the two vectors are scalar multiples and therefore identical in direction. Cosine correctly recognises that they encode the same topic distribution; the dot product, sensitive to magnitude, would mislead a length-sensitive ranker.

Example 2 (Ranking by similarity — Medium). A query $\mathbf{q} = (1, 1, 0, 0)^\top$ is compared against three document vectors:

$$\mathbf{d}_1 = (1, 1, 1, 0)^\top, \quad \mathbf{d}_2 = (2, 0, 0, 0)^\top, \quad \mathbf{d}_3 = (1, 1, 1, 1)^\top.$$

Rank the documents by cosine similarity to $\mathbf{q}$.

Solution. $\|\mathbf{q}\| = \sqrt{2}$, $\|\mathbf{d}_1\| = \sqrt{3}$, $\|\mathbf{d}_2\| = 2$, $\|\mathbf{d}_3\| = 2$. $\mathbf{q}^\top \mathbf{d}_1 = 2$, $\mathbf{q}^\top \mathbf{d}_2 = 2$, $\mathbf{q}^\top \mathbf{d}_3 = 2$.

$$\cos(\mathbf{q}, \mathbf{d}_1) = 2/(\sqrt{2}\sqrt{3}) \approx 0.816, \quad \cos(\mathbf{q}, \mathbf{d}_2) = 2/(\sqrt{2} \cdot 2) = 1/\sqrt{2} \approx 0.707,$$

$$\cos(\mathbf{q}, \mathbf{d}_3) = 2/(\sqrt{2} \cdot 2) \approx 0.707.$$

Ranking: $\mathbf{d}_1 \succ \mathbf{d}_2 \approx \mathbf{d}_3$. $\mathbf{d}_1$ is closest to $\mathbf{q}$ because both query terms appear and the document is short. $\mathbf{d}_2$ overweights one term; $\mathbf{d}_3$ contains both query terms but also two off-topic ones, diluting the cosine.

Example 3 (Cosine vs Euclidean for clustering — Hard). Argue why k -means clustering in TF-IDF space typically requires *prior* ℓ_2 -normalisation of the vectors. What pathology arises if normalisation is skipped?

Solution. k -means minimises within-cluster sum of squared Euclidean distances. Without normalisation, document magnitudes dominate the distance: long documents have large $\|\mathbf{x}\|$, hence large $\|\mathbf{x} - \boldsymbol{\mu}\|$ regardless of topic. As a result, the algorithm tends to form clusters by length rather than by topic — precisely the wrong outcome.

After ℓ_2 -normalisation, all vectors lie on the unit sphere; Euclidean distance becomes a monotone function of cosine similarity (Property 2.4.2), and k -means is equivalent to spherical k -means, which clusters by topic regardless of length. Practitioners therefore always normalise before k -means in NLP.

2.4.9 Python snippet: cosine, Euclidean, and FAISS

```
1 import numpy as np
2 from sklearn.feature_extraction.text import TfidfVectorizer
3 from sklearn.metrics.pairwise import cosine_similarity,
4   euclidean_distances
5
6 corpus = [
7     "Tunisia_football_national_team_won_the_cup",
8     "the_cup_of_tea_I_had_this_morning_was_great",
9     "national_football_leagues_across_Africa",
10    "great_cup_of_coffee_at_this_Tunis_cafe",
11 ]
```

```

11
12 vec = TfidfVectorizer().fit(corpus)
13 X = vec.transform(corpus)
14
15 q = vec.transform(["Tunisia_football"])
16
17 print("Cosine_similarities_to_query:")
18 for i, s in enumerate(cosine_similarity(q, X).ravel()):
19     print(f"doc_{i}: {s:.3f} -- {corpus[i]}")
20
21 print("\nEuclidean_distances_to_query:")
22 for i, s in enumerate(euclidean_distances(q, X).ravel()):
23     print(f"doc_{i}: {s:.3f} -- {corpus[i]}")
24
25 # Equivalent dot product after L2-normalisation
26 from sklearn.preprocessing import normalize
27 Xn = normalize(X, norm="l2")
28 qn = normalize(q, norm="l2")
29 print("\nDot_product_on_normalised_vectors_(==_cosine):")
30 print(np.asarray(qn @ Xn.T.toarray()).ravel().round(3))

```

For a million-document index, replace these dense methods by an approximate-nearest-neighbour engine (e.g. `faiss.IndexFlatIP` or `IndexHNSWFlat`).

2.4.10 Exercises

1. Compute, by hand, the cosine similarity, dot product, and Euclidean distance between

$$\mathbf{a} = (1, 2, 0, 1)^\top, \quad \mathbf{b} = (2, 4, 0, 2)^\top, \quad \mathbf{c} = (0, 1, 1, 1)^\top.$$

Comment on the relative ranking of $\mathbf{a}$ to $\mathbf{b}$ vs $\mathbf{a}$ to $\mathbf{c}$ under each measure.

2. Show, using only the definition, that $\cos(\mathbf{a}, \mathbf{a}) = 1$ for every non-zero vector $\mathbf{a}$, and that $\cos(\mathbf{a}, -\mathbf{a}) = -1$.
3. Prove that the angular distance $\theta(\mathbf{a}, \mathbf{b}) = \arccos(\cos(\mathbf{a}, \mathbf{b}))$ satisfies the triangle inequality on the unit sphere. (Sketch the geometric argument; full proof requires spherical geometry.)
4. Why is the dot product the preferred similarity in production search systems built on dense embeddings? (Hint: think about ℓ_2 -normalisation and hardware.)
5. (Synthesis.) Sketch a Tunisian-news semantic search engine: which embedding model would you use, what similarity measure, what indexing approach (exact vs approximate), and how would you evaluate retrieval quality? Identify two failure cases that purely lexical (TF-IDF) search would miss but a dense, multilingual model would handle.

2.5 Evaluation metrics

2.5.1 Motivation

“*What is not measured cannot be improved.*” An NLP system without principled evaluation is an NLP system without progress. Different tasks demand different metrics: a classifier is judged on whether it returns the correct label, a translation system on how closely its output matches a reference, a language model on how well it predicts unseen text. This section formalises the metrics most often quoted in the literature: **Accuracy**, **Precision**, **Recall**, F_1 for classification (Section 2.5.3), and **Perplexity**, **BLEU**, **ROUGE** for generation (Section 2.5.4). We pay equal attention to their pitfalls — a single number is never the whole story.

2.5.2 Approach by problem: the misleading 95% accuracy

Problem. You build a classifier that detects whether a tweet contains hate speech. Hate-speech tweets account for 5% of your corpus. A trivial baseline that always predicts *not hate speech* achieves 95% accuracy. Is this baseline useful? What metric reveals its uselessness?

Solution sketch. The baseline misses every actual case of hate speech — its *recall on the positive class* is 0. Accuracy is misleading whenever classes are imbalanced: it is dominated by the easy majority class. Reporting *precision and recall on the positive class*, or the F_1 score, makes the baseline’s failure visible immediately. This single example motivates everything that follows.

2.5.3 Classification metrics

The confusion matrix

Definition 2.5.1 (Confusion matrix, binary case). For a binary classifier with output $\hat{y} \in \{0, 1\}$ and ground-truth label $y \in \{0, 1\}$, define four counts on a test set of size N :

	Predicted positive	Predicted negative
Actual positive	TP (true positive)	FN (false negative)
Actual negative	FP (false positive)	TN (true negative)

By construction, $TP + FP + FN + TN = N$.

Accuracy

Definition 2.5.2 (Accuracy).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

The fraction of predictions that match the ground truth.

Accuracy is intuitive but misleading under class imbalance. Use it only when the class distribution is roughly balanced and the costs of false positives and false negatives are comparable.

Precision and Recall

Definition 2.5.3 (Precision and Recall).

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Library analogy. Imagine searching a library for books on machine learning.

- **Precision:** of the books you brought home, how many are about ML?
- **Recall:** of all ML books in the library, how many did you bring home?

A search that returns only the most obvious ML book has high precision but low recall. A search that returns every book in the library has perfect recall but very low precision.

When to optimise which.

- **Precision-critical:** spam detection (a false positive blocks a legitimate email — much worse than letting one spam through).
- **Recall-critical:** medical screening (missing a tumour is far worse than a false alarm — patients can be re-tested).
- **Balanced:** most generic NLP tasks (NER, sentiment, topic classification).

F-measure

Definition 2.5.4 (F_1 score). The F_1 score is the harmonic mean of Precision and Recall:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

The harmonic mean penalises imbalance more harshly than the arithmetic mean: F_1 is high only when *both* Precision and Recall are high.

Property 2.5.1 (Bounds of F_1). $0 \leq F_1 \leq 1$. $F_1 = 1$ iff Precision = Recall = 1. If either is zero, $F_1 = 0$.

Remark 2.5.1 (F_β generalisation). The general F_β score weights recall β times more than precision:

$$F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}.$$

F_2 favours recall (medical), $F_{0.5}$ favours precision (anti-spam).

Multi-class extensions

For K -class problems, compute Precision/Recall/ F_1 per class, then aggregate:

- **Macro-average:** simple average of the K per-class metrics. Treats all classes equally.
- **Micro-average:** pool all decisions across classes, then compute the metric. Dominated by frequent classes.
- **Weighted average:** weight each class by its prevalence.

2.5.4 Generation metrics

Perplexity

Generation models do not produce a discrete label; they produce a probability distribution over the next token. *Perplexity* measures how well the model predicts a held-out test sequence.

Definition 2.5.5 (Perplexity). For a language model P_θ and a sequence $w_1, \dots, w_N$ from a held-out set, the perplexity is

$$\text{PPL}(P_\theta) = 2^{-\frac{1}{N} \sum_{i=1}^N \log_2 P_\theta(w_i | w_1, \dots, w_{i-1})} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P_\theta(w_i | w_1, \dots, w_{i-1}) \right).$$

Intuition. If perplexity is k , the model is on average “hesitating between k equally likely words” at each position. Lower is better; perplexity of 1 means perfect prediction.

Property 2.5.2 (Lower bound). $\text{PPL}(P_\theta) \geq 1$, with equality iff P_θ assigns probability 1 to every observed token.

Remark 2.5.2 (A caveat). Low perplexity does not guarantee high-quality output. A model can have low perplexity on test text yet produce repetitive or incoherent generation. Perplexity is therefore a useful but partial proxy.

BLEU (translation)

Definition 2.5.6 (BLEU score, simplified). The Bilingual Evaluation Understudy score (Papineni et al., 2002) compares a candidate translation c to one or several reference translations $\{r_1, \dots, r_M\}$. It combines a *modified n -gram precision* p_n for $n = 1, \dots, N$ (typically $N = 4$) and a *brevity penalty* BP:

$$\text{BLEU} = \text{BP} \cdot \exp \left(\sum_{n=1}^N \frac{1}{N} \log p_n \right),$$

where

$$\text{BP} = \begin{cases} 1 & \text{if } |c| > |r| \\ \exp(1 - |r|/|c|) & \text{otherwise} \end{cases}$$

penalises candidates shorter than the reference.

Modified n -gram precision. For each n -gram in the candidate, count its occurrences in the candidate (call it count_c) and clip it by the maximum occurrences across the references. Sum the clipped counts and divide by the total candidate n -grams.

Range. $0 \leq \text{BLEU} \leq 1$ (often reported as a percentage). Modern strong systems on news translation reach 30–50 BLEU; human translation typically scores in the same range, illustrating that BLEU is a useful but imperfect proxy for human judgement.

ROUGE (summarization)

Definition 2.5.7 (ROUGE-N, simplified). For a candidate summary c and a reference r ,

$$\text{ROUGE-N} = \frac{|n\text{-grams}_c \cap n\text{-grams}_r|}{|n\text{-grams}_r|} = \text{Recall}_n.$$

ROUGE-1 uses unigrams; ROUGE-2 uses bigrams.

Definition 2.5.8 (ROUGE-L). ROUGE-L uses the length of the longest common subsequence (LCS) between c and r :

$$\text{ROUGE-L} = \frac{\text{LCS}(c, r)}{\max(|c|, |r|)}.$$

BLEU vs ROUGE. BLEU is precision-oriented (“what fraction of the candidate is correct?”); ROUGE is recall-oriented (“what fraction of the reference is captured?”). For *translation* we typically want both, but with a strong precision bias — hence BLEU. For *summarisation* we want to avoid missing important content, so recall is paramount — hence ROUGE.

2.5.5 Summary table

Metric	Task	Captures	Range
Accuracy	Classification	Overall correctness	$[0, 1]$
Precision	Classification	Quality of positive predictions	$[0, 1]$
Recall	Classification	Coverage of positive cases	$[0, 1]$
F_1	Classification	Harmonic balance of P and R	$[0, 1]$
Perplexity	Language modelling	Predictive quality (geometric mean)	$[1, \infty)$
BLEU	Translation	n -gram precision with brevity penalty	$[0, 1]$
ROUGE-N	Summarisation	n -gram recall vs reference	$[0, 1]$
ROUGE-L	Summarisation	Longest common subsequence recall	$[0, 1]$

Remark 2.5.3 (Modern complementary metrics). Each of the above is purely surface-form: it counts n -gram or token matches and ignores meaning. Modern alternatives such as *BERTScore* (semantic similarity via contextual embeddings), *LLM-as-a-judge* (a strong language model rates the candidate), *RAGAS* (evaluation suite for retrieval-augmented generation) and *MT-Bench* (multi-turn instruction-following) attempt to measure semantic and pragmatic adequacy. We encounter them in later parts of the course.

2.5.6 Worked examples

Example 1 (Confusion matrix — Easy). A spam detector tested on 1000 emails (300 spam, 700 legitimate) produces 250 true positives, 50 false negatives, 40 false positives, 660 true negatives. Compute Accuracy, Precision, Recall, and F_1 for the spam class.

Solution.

$$\text{Accuracy} = (250 + 660)/1000 = 0.91,$$

$$\text{Precision} = 250/(250 + 40) \approx 0.862,$$

$$\text{Recall} = 250/(250 + 50) \approx 0.833,$$

$$F_1 = \frac{2 \cdot 0.862 \cdot 0.833}{0.862 + 0.833} \approx 0.847.$$

Example 2 (Imbalanced classes — Medium). On a corpus where 99% of tweets are non-toxic, a classifier predicts *non-toxic* for every input. Compute its accuracy, precision, recall, and F_1 on the toxic class.

Solution. TP = 0, FN = 1% of corpus, FP = 0, TN = 99% of corpus. Accuracy = 0.99. Precision = 0/0 (undefined; conventionally 0). Recall = 0/1% = 0. F_1 = 0. The accuracy of 99% celebrates a model that detects no toxicity at all. F_1 correctly reveals it as useless on the task we care about. *Always inspect class-conditional metrics under imbalance.*

Example 3 (BLEU vs ROUGE on summary — Hard). Reference: “*Tunisia won the African Cup of Nations.*” (8 tokens) Candidate: “*Tunisia won the cup.*” (4 tokens)

(a) Compute the unigram precision of the candidate (BLEU-1, no brevity penalty for now). (b) Compute ROUGE-1. (c) Compute the brevity penalty and the full BLEU-1.

Solution. (a) Candidate unigrams: {Tunisia, won, the, cup}. All four appear in the reference. Modified precision = 4/4 = 1.0.

(b) ROUGE-1 unigram recall: reference unigrams (after lowercasing) {tunisia, won, the, african, cup, of, nations}, 7 distinct. Candidate covers {tunisia, won, the, cup}, 4 tokens. ROUGE-1 = 4/7 $\approx$ 0.571.

(c) Brevity penalty: $|c| = 4 < |r| = 8$, so BP = $\exp(1 - 8/4) = \exp(-1) \approx 0.368$. BLEU-1 = $0.368 \times 1.0 \approx 0.368$.

The candidate has high precision (everything it says is correct) but low recall (it omits half the reference). BLEU’s brevity penalty captures part of this issue; ROUGE captures it directly through recall.

2.5.7 Python snippet: classification + generation metrics

```

1  # Classification
2  from sklearn.metrics import precision_recall_fscore_support,
   classification_report
3
4  y_true = [1, 1, 1, 0, 0, 0, 1, 0]
5  y_pred = [1, 0, 1, 0, 1, 0, 1, 0]
6
7  p, r, f1, _ = precision_recall_fscore_support(
8      y_true, y_pred, pos_label=1, average="binary",
9  )
10 print(f"Precision={p:.3f}, Recall={r:.3f}, F1={f1:.3f}")
11 print("\nFull report:")
12 print(classification_report(y_true, y_pred, target_names=["neg", "
   pos"]))
13
14 # BLEU and ROUGE
15 # pip install nltk rouge-score
16 from nltk.translate.bleu_score import sentence_bleu,
   SmoothingFunction
17 from rouge_score import rouge_scorer
18
19 reference = "Tunisia won the African Cup of Nations".split()
20 candidate = "Tunisia won the cup".split()
21

```

```

22 bleu = sentence_bleu(
23     [reference], candidate,
24     weights=(1.0, 0, 0, 0), # BLEU-1
25     smoothing_function=SmoothingFunction().method1,
26 )
27 print(f"\nBLEU-1={bleu:.3f}")
28
29 scorer = rouge_scorer.RougeScorer(["rouge1", "rougeL"], use_stemmer
    =True)
30 scores = scorer.score(" ".join(reference), " ".join(candidate))
31 for k, v in scores.items():
32     print(f"_{k}:_P={v.precision:.3f}_R={v.recall:.3f}_F={v.
        fmeasure:.3f}")

```

2.5.8 Exercises

1. For a NER tagger that finds 80 true entities, misses 20, and produces 30 spurious entities, compute Precision, Recall, and F_1 on the entity class.
2. Show that for any binary classifier $F_1 \leq \min(\text{Precision}, \text{Recall})$, with equality iff Precision = Recall.
3. A language model assigns probabilities 0.5, 0.5, 0.25 to the three tokens of a held-out sequence. Compute its perplexity. What perplexity would a uniform model over a vocabulary of $|V| = 10,000$ achieve?
4. For the reference “*The cat is on the mat*” and the candidate “*The cat sits on the mat*”, compute BLEU-2 and ROUGE-2 by hand. What linguistic phenomenon is being measured (or missed) by each?
5. (Synthesis.) Design an evaluation protocol for a Tunisian-Arabic dialect-to-French translation system. Identify which metrics you would report, which biases each metric carries (e.g. BLEU’s surface-form sensitivity), and how you would complement them with human evaluation.